



MINISTERUL EDUCAȚIEI ȘI CERCETĂRII
UNIVERSITATEA PETROL – GAZE DIN PLOIEȘTI
FACULTATEA: LITERE ȘI ȘTIINȚE
DEPARTAMENTUL: INFORMATICĂ, TEHNOLOGIA INFORMAȚIEI,
MATEMATICĂ ȘI FIZICĂ
PROGRAMUL DE STUDII: INFORMATICĂ
FORMA DE ÎNVĂȚĂMÂNT: IF

LUCRARE DE LICENȚĂ

Conducător științific:
Lect. dr. inf. Șchiopu Daniela

Absolvent:
Coșereanu Emanuel

PLOIEȘTI
2026

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII
UNIVERSITATEA PETROL – GAZE DIN PLOIEȘTI
FACULTATEA: LITERE ȘI ȘTIINȚE
DEPARTAMENTUL: INFORMATICĂ, TEHNOLOGIA INFORMAȚIEI,
MATEMATICĂ ȘI FIZICĂ
PROGRAMUL DE STUDII: INFORMATICĂ
FORMA DE ÎNVĂȚĂMÂNT: IF

Vizat
Facultatea Litere și Științe

Aprobat,
Director de departament,
Lector dr. Anca Baci

LUCRARE DE LICENȚĂ

Aplicație software cu jocuri retro – Retro Games

Conducător științific:
Lect. dr. inf. Șchiopu Daniela

Absolvent:
Coșereanu Emanuel

PLOIEȘTI
2026



UNIVERSITATEA PETROL - GAZE DIN PLOIESTI

FACULTATEA: LITERE ȘI ȘTIINȚE

DOMENIUL: INFORMATICĂ

PROGRAMUL DE STUDII: INFORMATICĂ

FORMA DE ÎNVĂȚĂMÂNT: IF/FR/ID: IF

Anexa 7

Aprobat, Director de departament, Lector dr. Anca Baci	Declar pe propria răspundere că voi elabora personal lucrarea de licență și nu voi folosi alte materiale documentare în afara celor prezentate la capitolul „Bibliografie”. Semnătură student:
DATELE INIȚIALE PENTRU LUCRARE LICENȚĂ	
Proiectul a fost dat studentului: Coșereanu Emanuel	
1) Tema proiectului	
Aplicație software cu jocuri retro – Retro Games	
2) Data eliberării temei: Septembrie 2025	
3) Tema a fost primită pentru îndeplinire la data: Noiembrie 2025	
4) Termenul pentru predarea lucrării: Iulie 2026	
5) Elementele inițiale pentru lucrare:	
Cursul de Dezvoltarea aplicațiilor Web	
Bibliografie	
6) Enumerarea problemelor care vor fi dezvoltate:	
Introducere, Istoria jocurilor, Tehnologii utilizare în cadrul lucrării, Dezvoltarea platformei de jocuri retro „Retro Games”, Ghidul utilizatorului, Concluzii și dezvoltări ulterioare	
7) Enumerarea materialului grafic:	
48 figuri	
8) Consultații pentru lucrare, cu indicarea părților din proiect care necesită consultarea:	
Periodic	
Conducător științific:	Student:
Lect. dr. inf. Șchiopu Daniela	Coșereanu Emanuel
Semnătura:	Semnătura:

CUPRINS

1	Introducere	7
1.1	Motivația alegerii temei	7
1.2	Obiectivele lucrării	7
1.3	Structura lucrării	8
2	Istoria jocurilor.....	9
2.1	Istoria jocurilor de tip puzzle	9
2.2	Istoria jocurilor 2D.....	11
2.3	Jocuri puzzle 2D care au inspirat dezvoltarea platformei Retro Games	13
2.3.1	Snake	13
2.3.2	Tetris	14
2.3.3	2048	14
2.3.4	Flow Free	15
2.4	Analiza comparativă a jocurilor	16
3	Tehnologii utilizate în cadrul lucrării.....	17
3.1	Visual Studio Code	17
3.2	Angular.....	17
3.2.1	TypeScript.....	18
3.2.2	HTML și CSS	18
3.3	Rider	18
3.4	C#	19
3.5	MySQL.....	19
3.6	Bootstrap	19
4	Dezvoltarea platformei de jocuri retro „Retro Games”	21
4.1	Analiza cerințelor aplicației	21
4.2	Proiectarea aplicației	22
4.3	Pregătirea mediului de lucru	23
4.3.1	Pregătirea mediului de lucru pentru partea de frontend	23
4.3.2	Pregătirea mediului de lucru pentru partea de backend	25
4.4	Paginile de start, login și register	26
4.5	Pagina principală	28
4.6	Snake	28
4.6.1	Modurile de joc.....	28
4.6.2	Tabla, șarpele și mărul.....	29
4.6.3	Logica de funcționare a jocului	30
4.6.4	Butoanele utilizate în joc	32
4.6.5	Clasamentul jucătorilor (top 5). Informațiile suplimentare	32
4.7	Tetris.....	34
4.7.1	Tabla, piesele (tetromino), următoarea piesă	34

4.7.2	Logica de funcționare a jocului	36
4.7.3	Butoanele utilizate în joc. Clasamentul jucătorilor (top 5). Alte mesaje pentru utilizator... ..	40
4.7.4	Secțiunea afișării piesei următoare „Next Piece”	41
4.8	Brick Breaker și Brick Breaker 2	42
4.8.1	Logica de funcționare a jocurilor.....	42
4.8.2	Ce aduce în plus Brick Breaker 2.....	44
4.9	Goal Scorer.....	45
4.10	Super Bricks	46
4.11	Pagina Rankings (Clasamente)	48
4.12	Paginile Chat și Report Bugs.....	49
4.13	Testarea și optimizarea aplicației	51
5	Ghidul utilizatorului	52
5.1	Înregistrare. Autentificare. Deconectare din Aplicație	52
5.2	Controalele jocurilor	52
5.2.1	Controale Snake.....	52
5.2.2	Controale Tetris.....	53
5.2.3	Controale Brick Breaker 1 și 2.....	53
5.2.4	Controale Goal Scorer	53
5.2.5	Controale Super Bricks	53
5.3	Scopul fiecărui joc.....	53
5.4	Pierderea vieților și pierderea jocului.....	54
5.5	Paginile Chat, Rankings și Report Bugs	54
6	Concluzii și dezvoltări ulterioare	56
	Declarație de transparență.....	57
	Bibliografie	58
	Anexe	i
	Anexa 1. Codul sursă pentru pagina de înregistrare (register.ts).....	i
	Anexa 2. Codul sursă pentru pagina principală (games-page.ts)	i
	Anexa 3. Codul pentru funcția de reșezare a mărului (placeApple())	iii
	Anexa 4. Funcția folosită pentru a obține culoarea următoarei piese în jocul Tetris (getNextPieceColor())	iii
	Anexa 5. Codul funcțiilor pentru pornirea și setarea intervalului în jocul Tetris (startInterval() și setInterval()).....	iii
	Anexa 6. Codul funcției folosite pentru rotirea tetromino-urilor în sensul acelor de ceasornic (movePieceClockwise()).....	iv
	Anexa 7. Codul funcției folosite pentru generarea unei noi piese în jocul tetris (generateRandomPiece())	v
	Anexa 8. Codul folosit pentru a asculta evenimentele de la tastatură în jocul Tetris (HostListener())	vii
	Anexa 9. Codul metodei ce verifică coliziunea dintre minge și portar în jocul Goal Scorer (GoalkeeperCollision()).....	viii

Anexa 10. Codul metodei ce mută rândurile de asteroizi cu un rând mai jos în jocul Super Bricks (moveWallsDown())X

Listă de figuri

Fig. 1 - Hangman[27]	9
Fig. 2 - Space Panic[19]	10
Fig. 3 - Lode Runner[28].....	10
Fig. 4 - Door Door[8]	11
Fig. 5 - Spacewar![21].....	11
Fig. 6 - Atari Video Computer System (VCS)[12]	12
Fig. 7 - Space Invaders[26]	12
Fig. 8 - Pac-Man[24]	12
Fig. 9 - Nintendo Entertainment System[12].....	13
Fig. 10 - Consola portabila „Game Boy”[12].....	13
Fig. 11 - Snake[23].....	14
Fig. 12 - Tetris[16]	14
Fig. 13 – Jocul 2048[25].....	15
Fig. 14 - Flow Free[14]	15
Fig. 15 - Diagrama UML pentru clase.....	23
Fig. 16 - Structura proiectului Retro Games (frontend).....	25
Fig. 17 - Structura proiectului Retro Games (backend).....	26
Fig. 18 - Pagina de start (prima variantă)	27
Fig. 19 - Pagina de start (a doua variantă).....	27
Fig. 20 - Pagina principală (prima variantă).....	28
Fig. 21 - Pagina principală (a doua variantă).....	28
Fig. 22 - Tabla de joc, șarpele și mărul (prima versiune)	29
Fig. 23 - Table de joc, șarpele și mărul (a doua versiune)	29
Fig. 24 - Butoanele jocului Snake (prima versiune).....	32
Fig. 25 - Butoanele jocului Snake (a doua versiune).....	32
Fig. 26 - Clasamentul și informațiile adiționale (prima versiune)	33
Fig. 27 - Clasamentul și informațiile adiționale (a doua versiune).....	34
Fig. 28 - Tabla de joc pentru Tetris (prima versiune).....	35
Fig. 29 - Tabla de joc pentru Tetris (a doua versiune).....	35
Fig. 30 - Clasamentul, butoanele, mesajul de încurajare (Tetris) (prima versiune).....	41
Fig. 31 - Clasamentul, butoanele, mesajul de încurajare (Tetris) (a doua versiune).....	41
Fig. 32 - Următoarea piesă, scorul, nivelul (prima versiune)	41
Fig. 33 - Următoarea piesă (a doua versiune).....	41
Fig. 34 - Jocul Brick Breaker 2 (prima versiune)	44
Fig. 35 - Jocul Brick Breaker 2 (a doua versiune).....	44
Fig. 36 - Jocul Goal Scorer (prima versiune)	45
Fig. 37 - Jocul Goal Scorer (a doua versiune).....	45
Fig. 38 - Jocul Super Bricks (prima versiune).....	47
Fig. 39 - Jocul Super Bricks (a doua versiune).....	47
Fig. 40 - Pagina Rankings (prima variantă).....	49
Fig. 41 - Pagina Rankings (a doua variantă).....	49
Fig. 42 - Pagina Chat (prima versiune).....	50
Fig. 43 - Pagina Chat (a doua versiune)	50
Fig. 44 - Mesajele de eroare salvate în baza de date.....	50
Fig. 45 - Butoanele pentru deconectare (a doua versiune)	52
Fig. 46 - Afișarea vieții în jocurile cu mai multe vieți.....	54
Fig. 47 - Butonul Refresh din pagina Chat.....	54
Fig. 48 - Butonul Games	55

Listă de tabele

Tabelul 1 Analiză comparativă între jocurile prezentate în lucrare	16
Tabelul 2 Modulile de joc, vizeta de deplasare a șarpelui și punctele obținute	28

1 INTRODUCERE

Tema centrală a proiectului „Retro Games” este readucerea în atenția adolescenților și tinerilor jocurile vechi, ale anilor '90; jocuri ce au contribuit la nașterea „gaming-ului” așa cum este cunoscut în prezent. Lucrarea este organizată în 6 capitole, care explică pe larg motivația alegerii temei, obiectivele aplicației, modul de creare a platformei și organizare a jocurilor, cât și modul de funcționarea a fiecărui joc și a platformei ca întreg.

Lucrarea prezintă conține 6 jocuri create și anume: Snake, Tetris, Brick Breaker 1 și 2, Goal Scorer și Super Bricks, fiecare dintre acestea urmărind trezirea anumitor emoții ce stau adormite în adâncul inimii fiecărui jucător, respectiv antrenarea minții și relația minte-corp prin supunerea acestora la provocări ce îi va forța să ia decizii rapide și eficiente pentru obținerea unui scor cât mai mare. Jocurile precum Snake, Tetris și Brick Breaker pun mare accent pe viteza de reacție, oferind un scor mult mai însemnat pentru nivele mai grele, în schimb ce Goal Scorer și Super Bricks se axează mai mult pe o gândire strategică, eficientă, calmă.

Pe lângă toate acestea, aplicația „Retro Games” vizează dezvoltarea culturală și socială a utilizatorilor prin paginile „About the Game” în care este prezentată o mică istorie a fiecărui joc, respectiv prin pagina „Chat” cu ajutorul căreia utilizatorii pot comunica unul cu celălalt.

1.1 MOTIVAȚIA ALEGERII TEMEI

Motivația creării aplicației web „Retro Games” a provenit din sarcina ce mi-a fost atribuită în timpul perioadei de practică, în vara dintre anii doi și trei de facultate, la firma TechBolide și anume creare unui joc de tip Snake folosind framework-ul Angular.

Pasiunea mea pentru jocurile ce nu implicau acțiune precum: fotbal, curse de mașini, lupte, ori strategie precum jocurile Clash of Clans și Clash Royale nu pot spune că era una de invidiat. Obişnuiam să văd jocurile de tip puzzle, ori jocurile 2D ca fiind plictisitoare, învechite, fără logică, singurul scop util al lor fiind pierdea timpului. Însă, după ce am pus bazele aplicației de la practică, am putut observa cum aceste jocuri sunt mai interesante decât par, iar pentru vremurile în care au apărut pentru prima dată, erau adevărate revoluții ale lumii gaming-ului, exact cum pentru vremurile noastre au fost jocuri precum Fortnite, GTA V (Grand Theft Auto V) sau PUBG (PlayerUnknown's Battlegrounds). Astfel, în mintea mea s-a produs un „click” care m-a ajutat să îmi aduc aminte de copilăria mea, petrecută jucându-mă „jocul cu șarpele”, așa cum îi spuneam noi atunci, ori încercând să deslușesc cum pot obține un scor mai mare în jocul „Tetris”, ori câte tipuri de jocuri și nivele sunt pe consola pe care ne-o cumpăram anual, în schimbul a 10 lei, de la bălciul ce se ținea în orașul Urlați, de Sfântul Ilie.

Cu amintirea unei copilării frumoase și dorința de a înțelege măcar acum, la 12 ani distanță cum funcționează aceste jocuri, și de a le prezenta sub o formă plăcută celor din aceeași generație cu mine, și nu numai, s-au pus bazele platformei de jocuri retro, „Retro Games”.

1.2 OBIECTIVELE LUCRĂRII

Misiunea proiectului „Retro Games” este de a readuce în lumina reflectoarelor jocurile vechi, într-un mod cât mai accesibil, competitiv și captivant pentru noua generație de tineri și adolescenți. Acest scop este dirijat de următoarele obiective specifice:

- ◆ Readucerea pe prima scenă a jocurilor simple de tip 2D.

- ◆ Creare unei platforme în care să se găsească într-un singur loc cât mai multe jocuri retro.
- ◆ Construirea unei aplicații care să le permită utilizatorilor nu doar să se joace, ci să și comunice unii cu alții, creând astfel o comunitate de oameni pasionați de jocuri de tip „brick puzzle”.
- ◆ Dezvoltarea unei aplicații ce stârnește competitivitatea prin prezența top-urilor și a clasamentelor.
- ◆ Elaborarea unei platforme care să nu se axeze doar pe divertisment, ci și pe dezvoltarea personală și culturală a jucătorilor prin dezvoltarea istoriei jocurilor.
- ◆ Cucerirea jucătorilor și inițializarea lor în lumea programării prin posibilitatea raportării erorilor, de orice tip, întâlnite pe parcursul experienței de joc.
- ◆ Crearea unui mediu de joc minimalist, pentru a facilita navigarea ușoară, cât și pentru a le permite utilizatorilor să se focuseze total asupra jocurilor.

1.3 STRUCTURA LUCRĂRII

Lucrarea este structurată în 6 capitole după cum urmează:

- ⇒ Capitolul 1, **Introducere**, ce prezintă tema lucrării, motivația din spatele proiectului și obiectivele spre care se aspiră pe parcursul dezvoltării aplicației.
- ⇒ Capitolul 2, **Istoria jocurilor**, în care se pune înaintea procesul de dezvoltare al jocurilor 2D, cât și istoria jocurilor de tip „brick puzzle”, așa cum au fost ele concepute inițial, respectiv prezentarea unor jocuri asemănătoare cu cele create în lucrare.
- ⇒ Capitolul 3, **Tehnologii utilizate în cadrul lucrării**, prezintă platformele Visual Studio Code și Rider, folosite pentru partea de frontend, respectiv backend a aplicației, framework-ul Angular și limbajele de programare: TypeScript, HTML, CSS și C#, cât și sistemul de gestionare al bazelor de date relaționare MySQL, respectiv framework-ul Bootstrap folosit pentru facilitarea stilizării părții de frontend.
- ⇒ Capitolul 4, **Dezvoltarea platformei de jocuri retro „Retro Games”** asigură o analiză amănunțită a etapelor de creare ale aplicației, ce debutează prin identificarea cerințelor, structurarea interfeței, a bazei de date, continuând prin implementarea în cod a fiecărui concept și funcționalitate, procesul de dezvoltare fiind încheiat printr-o testare riguroasă.
- ⇒ Capitolul 5, **Ghidul utilizatorului**, furnizează informații referitoare la pașii de înregistrare, conectare în aplicație, detaliază regulile fiecărui joc, respectiv controalele acestora, expunând instrucțiunile pentru utilizatori.
- ⇒ Capitolul 6, **Concluzii și dezvoltări ulterioare**, prezintă succesele obținute în cadrul procesului de dezvoltare al platformei, cât și sublinierea îndeplinirii Țelurilor formulate.

2 ISTORIA JOCURILOR

De-a lungul anilor, jocurile video au căpătat amploare în viața fiecărui individ, fie că acesta este un pasionat de jocuri, un adolescent care dorește să își petreacă timpul cu prietenii lui sau un părinte care aspiră la o relație părinte-copil mai bună, relație ce se poate îmbunătăți prin înțelegerea lumii copilului, și bineînțeles, a lumii jocurilor.

Jocurile nu au fost din totdeauna așa cum le cunoșteam noi astăzi, cu o grafică excelentă, cu o poveste foarte complex gândită, cu zeci de nivele și misiuni pentru a crea o experiență cât mai de neuitat pentru utilizator. Ele s-au dezvoltat treptat, încetul cu încetul, pornind de la jocuri 2D, de tip puzzle, ori cu grafică foarte simplă și ajungând în final la ceea ce cunoaștem în prezent și vedem cu ochii noștri: jocuri mai reale decât realitatea.

În acest capitol vom trece în revistă câteva momente importante, semnificative, de cotitură din istoria jocurilor 2D și de tip puzzle, pentru a înțelege mai bine de ce uneori e important să ne întoarcem la rădăcini pentru a nu ne pierde în necunoscut și a uita de unde am pornit.

2.1 ISTORIA JOCURILOR DE TIP PUZZLE

Jocurile de tip puzzle pun accentul pe dezvoltarea abilităților de rezolvare a unei probleme, de gândire logică și în prealabil, multe dintre ele cerând o gândire rapidă. Primele jocuri video de tip puzzle ce au fost implementate se bazează pe jocurile de gândire precum „Spânzurătoarea” (Fig. 1)[32], în care unul dintre jucători se gândește la un cuvânt sau frază, iar celălalt având obiectivul de a ghici prin sugerarea de litere, posedând un anumit număr de încercări [27].



The gallows is usually completed after 11 incorrect guesses, in this order:

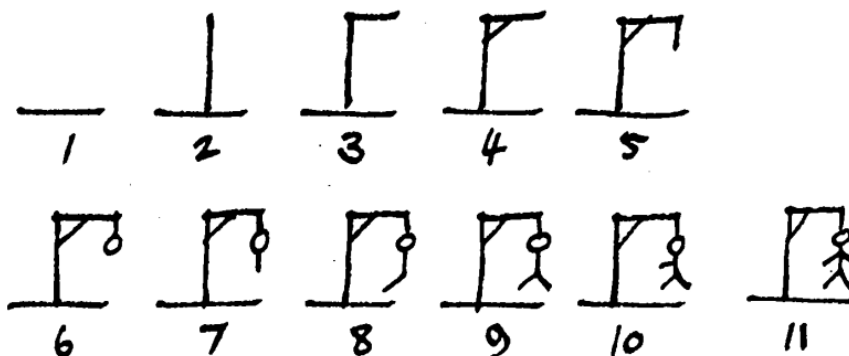


Fig. 1 - Hangman[27]

Un alt joc ce a reprezentat un jalon în istorie, fiind precursorul multor jocuri puzzle-platforms precum „Lode Runner” (Fig. 3) și „Door Door” (Fig. 4), (ambele create în 1983),

este „Space Panic” (**Fig. 2**) de la Universal Entertainment, lansat în sălile de jocuri arcade în 1980, în care jucătorul sapă gropi pentru a prinde creaturi [32], [19].

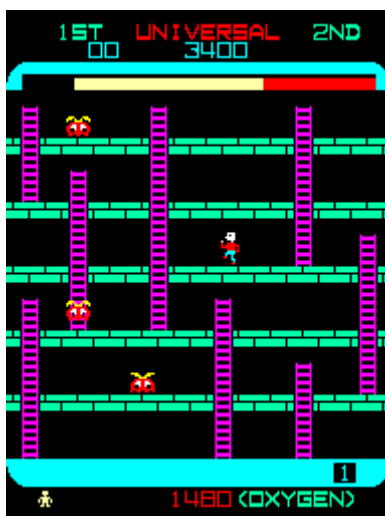


Fig. 2 - Space Panic[19]

„Lode Runner” este dezvoltat de Doug Smith și publicat de Broderbund în 1983. În cadrul acestui joc, jucătorul controlează un caracter ce trebuie să colecteze toate bucățile de aur dintr-un nivel și să ajungă la final în timp ce este urmărit de inamici. Jocul numără un total de 150 de nivele ce au ca obiectiv testarea abilităților de rezolvare a problemelor și de reacție a jucătorilor [32], [28].

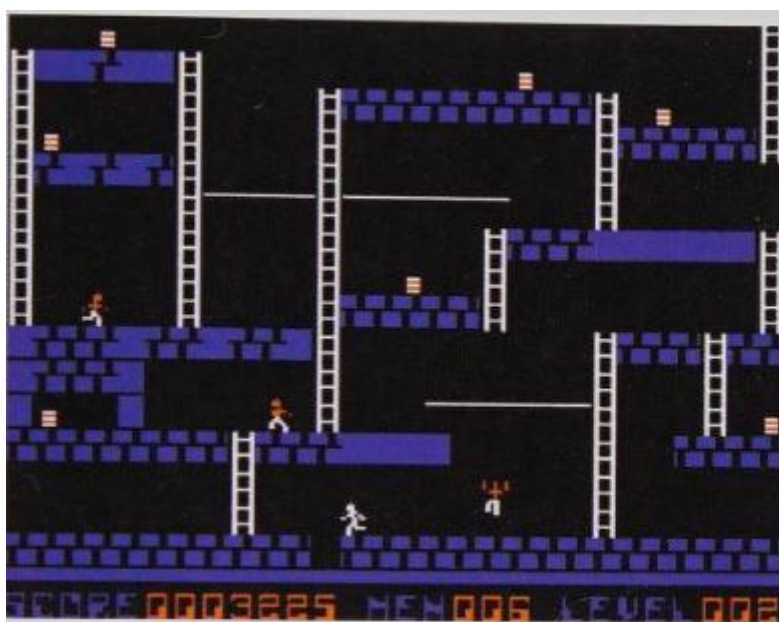


Fig. 3 - Lode Runner[28]

„Door Door” este un joc de tip puzzle publicat în Japonia în 1983. Jucătorii o controlează pe Chun, o creatură mică, în formă de ou, ce poartă o șapcă de baseball. Aceasta este fugărită continuu de extraterestrii ce o urmăresc folosind algoritmi determiniști. Cei mai previzibili dintre ei, Namegon și Amechan o urmăresc pe Chun pe cea mai directă cale posibilă, în schimb ce Invekun folosește scări pentru a urma căi ocolitoare, iar Otapyon o urmărește pe Chun imitându-i salturile [32], [8].

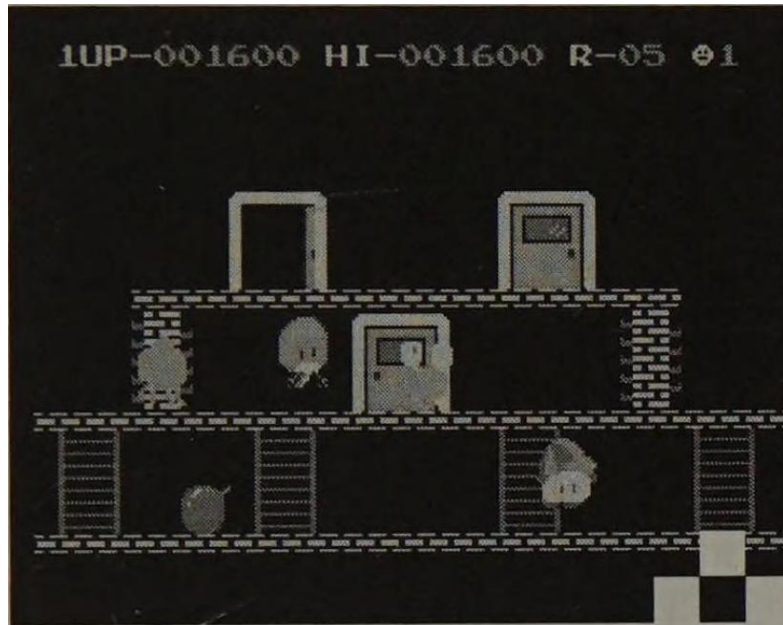


Fig. 4 - Door Door[8]

2.2 ISTORIA JOCURILOR 2D

Istoria jocurilor video are ca început anii 1950-1960 când informaticienii au început dezvoltarea jocurilor simple, respectiv a simulărilor pe minicomputere și pe computerele mainframe [30]. Unul dintre primele jocuri de acest gen este „Spacewar!” (Fig. 5), care prezintă două nave spațiale denumite „Acul” și „Pana” ce sunt angajate într-o luptă aeriană în timp ce zboară în câmpul gravitațional al unei stele, ambele fiind controlate de jucători umani [21].



Fig. 5 - Spacewar![21]

Până la mijlocul anilor 1970 microprocesoarele ieftine au înlocuit circuitele logice discrete tranzistor-tranzistor ale hardware-ului din acea vreme, iar primele console bazate pe casete ROM au apărut, incluzând „Atari Video Computer System (VCS)” (Fig. 6) [30], [12].



Fig. 6 - Atari Video Computer System (VCS)[12]

În timpul epocii de aur a jocurilor arcade, ce include „Space Invaders” (Fig. 7) și „Pac-Man” (Fig. 8) piața consolelor de jocuri video 2D a cunoscut o creștere formidabilă. Cu toate acestea, Statele Unite ale Americii a cunoscut în anul 1983 o cădere a industriei de jocuri video, cauzate în mare măsură de un potop de jocuri, majoritatea slabe, ori clonate, și de concurența venită din industria computerelor personale, și a noilor jocuri dezvoltate pentru ele [30].

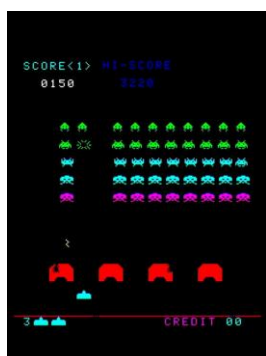


Fig. 7 - Space Invaders[26]



Fig. 8 - Pac-Man[24]

Prăbușirea din America a permis industriei din Japonia să preia conducerea pieței, culminând cu lansarea în anul 1985 a consolei „Nintendo Entertainment System (NES)” (Fig. 9) în Statele Unite, ajutând la relansare sectorului de jocuri video aflat în declin [30].

Finalul anilor 1980 și începutul anilor 1990 au cuprins jocuri video impulsionate de îmbunătățirile și standardizarea din sectorul computerelor personale și de competiția dintre marile companii ale timpului „Nintendo” și „Sega”, ce se luptau pentru piața din America [30].



Fig. 9 - Nintendo Entertainment System[12]

Primele console portabile memorabile au apărut în anii 1990, conduse de platforma „Game Boy” (Fig. 10) de la „Nintendo” [30].



Fig. 10 - Consola portabila „Game Boy”[12]

2.3 JOCURI PUZZLE 2D CARE AU INSPIRAT DEZVOLTAREA PLATFORMEI RETRO GAMES

Pe parcursul acestui subcapitol vor fi prezentate și analizate jocurile care au pus bazele platformei „Retro Games”, cât și alte jocuri puzzle asemănătoare. Fiecare dintre acestea vine cu noi provocări de gândire și dexteritate, cu un spirit competitiv, dar și cu o dorință de a aduce un colț de relaxare utilizatorului [31].

2.3.1 Snake

Jocul „Snake” (Fig. 11) este de departe unul dintre cele mai cunoscute, răspândite și iubite jocuri 2D, făcând parte din istoria de viață a fiecărui copil născut înainte de anii 2010. Tema jocului este una simplă, utilizatorul care conduce șarpele trebuie să mănânce cât mai multe mere fără a se lovi de el însuși (ori de pereți, în funcție de versiunea jocului). În cea mai mare parte a cazurilor, jocul este asociat cu telefonul mobil „Nokia 3310”, deoarece venea preinstalat pe acesta.



Fig. 11 - Snake[23]

2.3.2 Tetris

„Tetris” (**Fig. 12**) este un joc de tip puzzle dezvoltat în anul 1985 de către un inginer sovietic cu o serie de 220 de variante răspândite pe 70 de platforme. În cadrul jocului, piesele tetromino care cad și au forma literelor: O, T, Z, S, I, J, L, trebuie așezate într-o grămadă, fără spații libere. Odată ce o linie orizontală este umplută, aceasta va fi eliminată, generând puncte, respectiv, prevenind umplerea excesivă a grămezii. Piesele pot fi rotite pentru o mai bună potrivire în grămadă, iar la un moment dat, pot fi eliminate 1, 2, 3, sau chiar 4 rânduri o dată, generând mai multe puncte.



Fig. 12 - Tetris[16]

2.3.3 2048

Jocul „2048” (**Fig. 13**) a fost creat de dezvoltatorul web italian Gabriele Ciurilli și publicat pe GitHub, putând fi jucat doar de un singur jucător. Tema jocului este glisarea plăcilor pe care se află multiplii al lui 2 (2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048), astfel încât să se obțină numărul 2048. Cu toate acestea se poate continua jocul inclusiv după atingerea obiectivului, creând plăci cu numere mai mari.

„2048” este jucat pe o matrice de 4x4, pe care se află inițial 2 plăci cu valori de 2 sau 4. La fiecare glisare vor apărea noi plăci, ori cu valoarea 2, în proporție de 90%, ori cu valoarea 4, în proporție de 10%.

Placa cu cea mai mare valoare posibilă este 131072.

Jocul poate fi pierdut doar dacă plăcile nu se mai pot muta, ori combina [25].



Fig. 13 – Jocul 2048[25]

2.3.4 Flow Free

Scopul jocului „Flow Free” (**Fig. 14**) este de a conecta puncte de aceeași culoare, trasând „canale” între ele. Dificultatea este definită de mărimea tablei, care poate începe de la o matrice de 5x5, având 3 culori și poate ajunge la o matrice de 15x15, respectiv 16 culori. Unele nivele prezintă diverși pereți ce trebuie ocoliți pentru a unii punctele. Un nivel este câștigat atunci când toate punctele au fost unite cu succes [14].

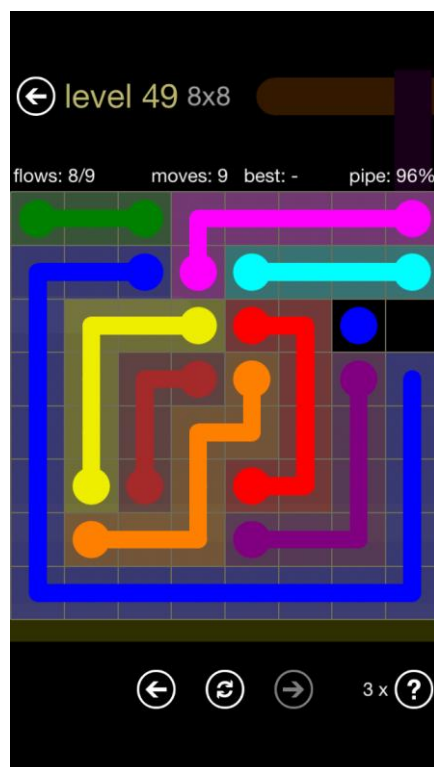


Fig. 14 - Flow Free[14]

2.4 ANALIZA COMPARATIVĂ A JOCURILOR

Pentru o mai bună înțelegere a atuurilor fiecărui joc prezentat în lucrare, a punctelor pe care acestea le accentuează în dezvoltarea gândirii, cât și a diferențelor dintre ele am realizat un tabel comparativ (Tabelul 1) pentru o viziune globală asupra caracteristicilor.

Tabelul 1 Analiză comparativă între jocurile prezentate în lucrare

Joc	Caracteristică principală	Tip de gândire solicitat	Dinamica dificultății
Snake	Simplitate și grafică minimalistă	Adaptare la mediu	Crește o dată cu dimensiunea șarpelui, nu pune accentul pe viteză.
Tetris	Rapiditate și eficiență	Gândire rapidă sub presiune	Orice greșeală este critică, punctajul crește în funcție de viteza de așezare
2048	Combinarea matematicii cu puzzle-ul	Gândire matematică și anticipare	Necesită planificare pe termen lung, fiind comparat cu jocurile de șah
Flow Free	Rezolvarea de nivele statice	Gândire de ansamblu (viziune globală)	Jucătorul trebuie să rezolve fiecare nivel, nu doar să reacționeze la elemente noi

3 TEHNOLOGII UTILIZATE ÎN CADRUL LUCRĂRII

Capitolul actual prezintă succint tehnologiile ce au stat la baza creării proiectului „Retro Games”, importanța utilizării fiecărui mediu și limbaj de programare, respectiv framework, cât și motivele din spatele alegerii fiecărei tehnologii. Mediile de dezvoltare folosite sunt Rider pentru backend și Visual Studio Code pentru frontend; framework-urile utilizate fiind Angular și Bootstrap, respectiv limbajele de programare C# și TypeScript; în plus s-a folosit HTML pentru structura paginilor web și CSS pentru stilizare.

3.1 VISUAL STUDIO CODE

Visual Studio Code este un editor de cod sursă creat de Microsoft pentru Windows, Linux, macOS și browsere web. Acesta poate fi utilizat pentru o varietate de limbaje de programare precum: C, C++, C#, Fortran, Go, Java, JavaScript, Python, Rust, motiv pentru care a și fost ales pentru dezvoltarea aplicației, pe lângă care se adaugă și mediul de dezvoltare simplu, dar eficient [33].

Caracteristici:

- Suport pentru depanare: Acesta dispune de un depanator încorporat, conceput pentru a îmbunătăți procesul de dezvoltare. Acesta oferă suport nativ pentru depanarea aplicațiilor Node.js, în timp ce depanatoarele pentru alte limbaje pot fi instalate prin extensii. Depanatorul le permite utilizatorilor să se atașeze la procesele care rulează și să parcurgă codul sursă linie cu linie în timpul execuției pentru a oferi o vizualizare detaliată a fluxului programului.
- Evidențierea sintaxei care determină culoarea și aspectul codului sursă afișat în editor. Aceasta are responsabilitatea de a colora diferit cuvintele cheie precum: if, for, while în JavaScript, ori tag-urile în HTML pentru o mai ușoară diferențiere între instrucțiuni, comentarii, numele variabilelor, etc.
- Controlul versiunilor încorporat cu Git: Visual Studio Code recunoaște când deschizi un folder care este un depozit Git și îți permite să utilizezi majoritatea operațiilor Git fără a utiliza linia de comandă [29].

3.2 ANGULAR

Angular este o platformă de dezvoltare web ce are la bază limbajul TypeScript, dezvoltată de către Google. Acesta este o rescriere completă a framework-ului AngularJS [5].

Caracteristici:

- Structurare pe componente. Acestea sunt principalele elemente ale aplicațiilor Angular, fiecare dintre ele având în componență: un șablon HTML, în care se definește ceea ce este afișat pe pagină, un selector CSS, în care se poate edita aspectul paginii, respectiv o clasă TypeScript care definește comportamentul [5].
- Rutare. Angular înglobează un router care permite dezvoltatorilor să definească și să gestioneze stările aplicațiilor și căile de navigare, făcând mai ușoară crearea unei pagini cu rutare complexă [5].

Caracteristicile menționate mai sus au fost și motivele principale pentru alegerea framework-ului Angular pentru dezvoltarea platformei de jocuri retro „Retro Games”.

3.2.1 TypeScript

TypeScript este un limbaj de programare de nivel înalt conceput pentru construirea aplicațiilor mari, dezvoltat de Microsoft [13].

Caracteristici:

- Superset al lui JavaScript ceea ce înseamnă că orice cod valid scris în JavaScript poate fi rulat și în TypeScript. Pe lângă aceasta, vine și cu alte caracteristici suplimentare.
- Static Typing: permite definirea tipurilor pentru variabile, parametrii funcțiilor, respectiv pentru valorile returnate.
- Type Interference: dacă tipul variabilei nu este declarat explicit, TypeScript îi va atribui automat tipul potrivit.
- Suportă programarea orientată obiect.

3.2.2 HTML și CSS

Hypertext Markup Language (HTML) este limbajul standard pentru documente destinate să fie afișate într-un browser web. Acesta definește cuprinsul și structura conținutului web, adesea fiind asistat de tehnologii precum Cascading Style Sheets (CSS) și limbaje de scripting ca JavaScript, respectiv TypeScript [6].

Cascading Style Sheets este un limbaj utilizat pentru stilizarea unui document scris într-un limbaj de marcare precum HTML sau XML. Este conceput pentru a permite separarea conținutului de prezentare, inclusiv aspectul, culorile și fonturile [17].

3.3 RIDER

JetBrains Rider este un „all-in-one” IDE (Integrated Development Environment) pentru dezvoltatorii care lucrează cu toată gama de tehnologii .NET, precum și pentru cei implicați în dezvoltarea de jocuri. Setul său de funcții permite dezvoltarea unei serii largi de aplicații ce vizează framework-uri precum .NET, ASP.NET, MAUI, dar și motoare de jocuri precum Unity, Unreal Engine sau Godot. Arhitectura Rider este concepută pentru viteză, și răspuns rapid, făcând foarte facilă întreținerea codului. Pe lângă acestea, oferă o experiență de dezvoltare excelentă, funcționând impecabil, atât pe Windows, cât și pe Linux sau macOS. Datorită acestor atuuri, Rider a fost ales pentru dezvoltarea părții de backend în cadrul proiectului „Retro Games” [15].

Alte caracteristici:

- Editor de cod inteligent. Rider oferă o mulțime de funcții inteligente de editare a codului printre care se numără: diferite tipuri de completare a codului, importul automat al spațiilor de nume, inserarea automată a acoladelor și evidențierea delimitatorilor potriviți, respectiv rearanjarea codului.
- Navigare și căutare. Accesarea rapidă a oricărui fișier, tip sau membru din cod folosind scurtătura „Navighează către”.

3.4 C#

C# este limbajul de programare ales pentru realizarea backend-ului aplicației. Acesta este un limbaj simplu, modern, ce implementează programarea orientată obiect preluată din C++ și Java [9] conceput pentru a fi potrivit atât pentru scrierea aplicațiilor pentru sisteme hosted cât și embedded [11]. Printre caracteristicile principale ale limbajului se află lipsa pointerilor, gestionarea automată a memoriei și „garbage collection”, variabilele de tip întreg 0 și 1 nu mai sunt acceptate ca variabile booleane, cele din urmă sunt acceptate doar ca fiind valori de tip „false” sau „true”; operatorul „==” este utilizat pentru compararea a doua valori, iar „=” este folosit pentru atribuirea unei valori, orice componentă poate fi transformată într-un „web service” ce poate fi accesat din orice aplicație rulând pe orice platformă, nu se mai pot efectua conversii nesigure precum transformarea din variabilă de tip double în variabilă de tip boolean [9].

Portabilitatea codului este un aspect foarte important, la fel ca și portabilitatea programatorului mai ales pentru cei deja obișnuiți cu C și C++ [11].

Aplicațiile C# sunt concepute să fie economice în ceea ce privește utilizarea memoriei și puterea de procesare, neputând concura direct în ceea ce privește performanța și dimensiunea cu aplicațiile create cu C sau cu limbajul de asamblare [11].

3.5 MySQL

MySQL este cel mai popular sistem open-source de gestiune a bazelor de date SQL, dezvoltat, distribuit și întreținut de către Oracle. Acesta lucrează cu baze de date relaționare. Serverul de baze de date MySQL este foarte rapid, de încredere, scalabil și ușor de utilizat. Inițial a fost dezvoltat pentru a gestiona baze de date mari mult mai rapid decât soluțiile existente și a fost utilizat cu succes în medii de dezvoltare extrem de dificile pentru mulți ani. Deși aflat sub o continuă dezvoltare, serverul MySQL oferă astăzi un set bogat și folositor de funcții. Conectivitatea, viteza și securitatea îl fac foarte potrivit pentru accesarea bazelor de date online. Acesta funcționează în sisteme client/server sau integrate [20].

Motivul pentru folosirea acestui sistem de gestiune pentru realizarea bazei de date a platformei „Retro Games” a fost simplitatea lui, faptul că este ușor de utilizat și înțeles, respectiv explorarea acestuia în timpul anilor de studiu din cadrul facultății.

3.6 BOOTSTRAP

Bootstrap este cel mai cunoscut framework frontend open-source folosit pentru proiectarea interfețelor web utilizând CSS și JavaScript, facilitând la stilizarea elementelor esențiale din cadrul siteurilor web precum barele de navigare, formularele și butoanele. Totodată Bootstrap este recunoscut pentru sistemul grilei receptive („Grid Layout”) care permite ajustarea aplicației web în funcție de mărimea ecranului dispozitivului [18].

Bootstrap este menținut de către o mică echipă de dezvoltatori de pe GitHub. Original a fost creat la Twitter la mijlocul anului 2010, cunoscut sub numele de *Twitter Blueprint*. După câteva luni Twitter a susținut primul său Hack Week (Săptămâna Hackingului) și dezvoltatori cu niveluri diferite de abilitate au dorit să se implice în îmbunătățirea acestuia. A fost utilizat drept ghid pentru dezvoltarea instrumentelor interne timp de peste un an înainte de a fi lansat public și continuă să fie și astăzi. Acesta a fost publicat pe data de 19 August 2011 [7].

Framework a fost utilizat în cadrul proiectului pentru a dezvolta mai eficient abilitatea elementelor siteului de a-și schimba aspectul în funcție de dimensiunea ecranului și pentru o mai bună stilizare a acestora.

4 DEZVOLTAREA PLATFORMEI DE JOCURI RETRO „RETRO GAMES”

În capitolul curent se va prezenta în detaliu fiecare pas, fiecare etapă, fiecare moment ce a dus la dezvoltarea platformei web de jocuri „Retro Games”, începând de la analiza cerințelor (cerințe ce au legătură cu experiența utilizatorului, respectiv cu stilul de joc), proiectarea aplicației, pregătirea mediului de lucru (crearea fișierelor necesare și ierarhizarea acestora) și încheind prin testarea, respectiv optimizarea acesteia pentru a crea o aplicație fără erori (sau cu cât mai puține erori posibil).

4.1 ANALIZA CERINȚELOR APLICAȚIEI

Pentru realizarea aplicației „Retro Games” s-a urmărit simplitatea și funcționalitatea, fapt ce se va putea observa și din cerințele ce vor fi prezentate mai jos. Scopul este de a crea o platformă cât mai prietenoasă cu utilizatorul (user-friendly), fără a-i crea probleme la utilizare.

Experiența utilizatorului (UX):

- *Înregistrare/autentificare:* La deschiderea aplicației, utilizatorul va fi redirecționat către pagina pentru înregistrare/autentificare, ce îi va oferi posibilitatea să aleagă dacă dorește crearea unui cont nou, ori autentificarea într-un cont existent.
- *Pagina de jocuri:* După autentificare utilizatorul va fi redirecționat către pagina cu jocuri (Snake, Tetris, Brick Breaker, Brick Breaker 2, Goal Scorer, Super Bricks), pe care se vor afla și butoanele pentru paginile de chat, report bugs, respectiv pentru a vedea clasamentele la fiecare joc și pentru a se deconecta.
- *Pagina cu clasamente:* În cadrul platformei va exista o pagină în care utilizatorul va putea vedea clasamentele pentru fiecare joc în parte.
- *Jocurile:* În cadrul aplicației vor fi prezentate 6 jocuri ce vor oferi o experiență de utilizare amplă.
- *Chat și Report Bugs:* Utilizatorilor li se va permite să comunice unii cu alții și să raporteze către dezvoltator erorile întâlnite în cadrul aplicației.
- *About the Game:* Fiecare joc va avea o pagină dedicată în care se vor explica regulile de joc (ex: cum se câștigă puncte), cum se joacă fiecare (ex: cum se controlează șarpele), respectiv o mică istorie a lui.
- *Ecranul fiecărui joc/nivel:* Fiecare joc va avea prezente 4 butoane ce îi va permite utilizatorului să se deconecteze, să navigheze către pagina anterioară, să pună jocul pe pauză, ori să îl reînceapă.

Gameplay:

- *Snake:* Jocul Snake va fi creat cu 3 nivele de joc, care se vor diferenția prin viteza de deplasare a șarpelui, plus un mod diferit de joc ce va permite șarpelui să treacă prin perete. Șarpele va fi controlat din săgețile: sus, jos, stânga, dreapta. Scopul acestui joc este obținerea a cât mai multor puncte prin mâncarea merelor.

- *Tetris*: Piesele de tetromino vor fi controlate din săgeți: sus – pentru a roti piesa, stângă/dreapta – pentru a le muta, jos – pentru ca piesa să coboare mai repede, respectiv tasta space ce permite implementarea hard drop-ului (piesa va cădea automat jos). Scopul jocului este de a crea cât mai multe linii pline, ce vor fi eliminate, pentru obținerea unui punctaj cât mai mare.
- *Brick Breaker* și *Brick Breaker 2*: Diferența dintre aceste două jocuri va fi prezența unui paddle (paletă) în partea de sus a tablei în cel de al doilea joc. Ambele vor veni cu 3 nivele care se vor diferenția prin viteza de mișcare a bilei, respectiv un mod de joc denumit „heart mode” în care cărămizile vor fi așezate în formă de inimă.
- *Goal Scorer*: Scopul jocului este învingerea portarului și înscrierea a cât mai multor goluri. Atacantul va fi controlat din săgețile stânga/dreapta, iar la apăsarea tastei space mingea se va mișca mai repede.
- *Super Bricks*: În cadrul acestui joc utilizatorul va trebui să controleze o rachetă pentru a crea cât mai multe linii pline de meteoriți ce vor fi eliminați. Racheta se va controla din săgețile stânga/dreapta, iar la apăsarea tastei space aceasta va trage.

4.2 PROIECTAREA APLICAȚIEI

Pentru realizarea platformei va fi nevoie de structurarea datelor într-un mod cât mai eficient, din acest motiv se vor crea tabele (modele) SQL pentru a stoca datele, controllere pentru a primi cererile de la utilizator și a comunica cu baza de date pentru a procesa informația, DTO-uri (Data Transfer Objects) pentru a muta datele între server și site, baza de date (ApplicationDbContext) pentru a asigura legătura între codul C# și tabelele SQL. Aceste elemente vor fi folosite în felul următor:

- Sistemul de autentificare va folosi modelul de utilizator și va genera un token de acces. Parolele vor fi criptate folosind Biblioteca BCrypt.
- Sistemul de clasament va căuta scorurile în baza de date și va calcula locul fiecărui jucător în timp real.
- Sistemul de chat și bug-uri va permite comunicarea și raportarea problemelor salvând momentul realizării fiecărei acțiuni.

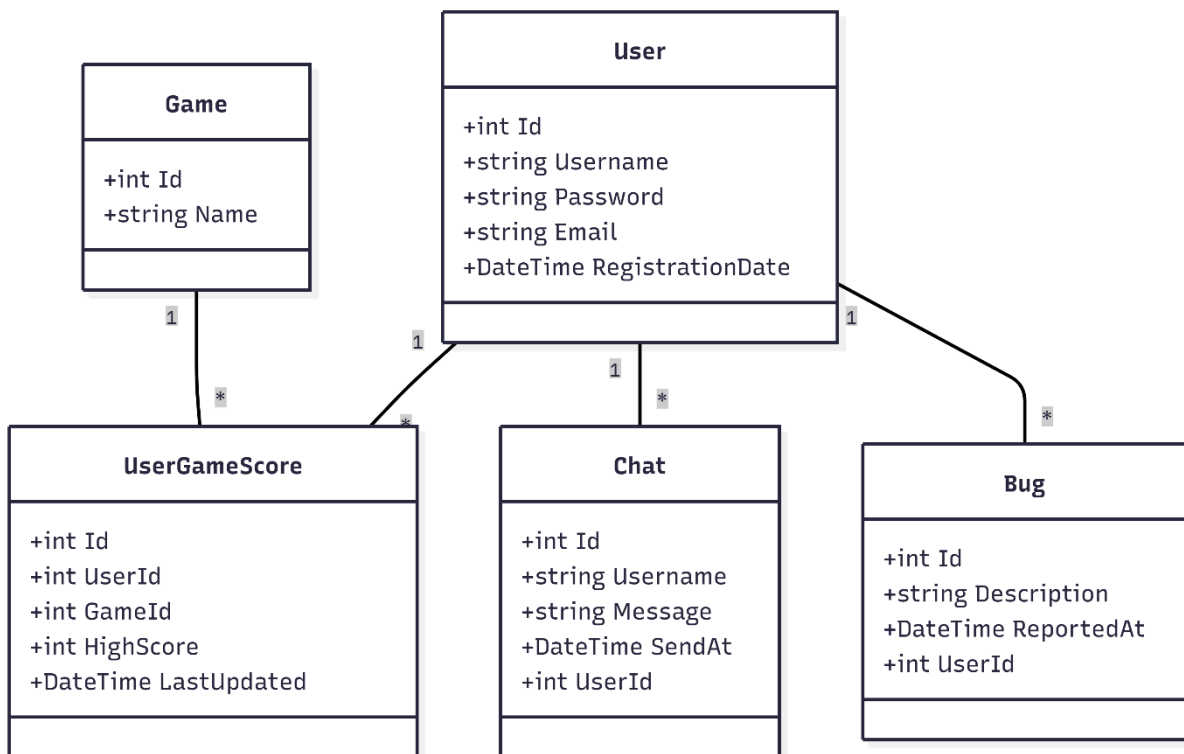


Fig. 15 - Diagrama UML pentru clase

4.3 PREGĂTIREA MEDIULUI DE LUCRU

Pregătirea mediului de lucru pentru începerea proiectului a însemnat instalarea aplicației Visual Studio Code și framework-ului Angular pentru partea de frontend, respectiv Rider și serverul WAMP pentru partea de backend. Totodată, în ambele aplicații s-a creat un nou proiect unde au fost organizate în mod ierarhic, pentru o mai bună structurare toate folderele și fișierele de care va fi nevoie pe parcursul creării aplicației.

4.3.1 Pregătirea mediului de lucru pentru partea de frontend

Pentru realizarea părții de frontend s-a creat proiectul folosind comanda „ng new retro_games”, iar mai apoi s-au creat componentele necesare pentru fiecare aplicație (**Fig. 16**, **Fig. 17**), prezentate mai jos:

- **src/app/about-games/about-brickbreaker** – componentă pentru pagina despre jocul Brick Breaker
- **src/app/about-games/about-brickbreakertwo** – componentă pentru pagina despre jocul Brick Breaker 2
- **src/app/about-games/about-goalscorer** – componentă pentru pagina despre jocul Goal Scorer
- **src/app/about-games/about-snake** – componentă pentru pagina despre jocul Snake
- **src/app/about-games/about-superbricks** – componentă pentru pagina despre jocul Super Bricks

- **src/app/about-games/about-tetris** – componentă pentru pagina despre jocul Tetris
- **src/app/chat** – componentă folosită pentru crearea paginii de chat
- **src/app/games/brickbreaker/brickbreaker-start-page** – componentă folosită pentru a crea pagina de start a jocului Brick Breaker
- **src/app/games/brickbreaker/brickbreaker1** (**brickbreaker2, brickbreaker3, brickbreaker-heart**) – componente folosite pentru nivele jocului Brick Breaker
- **src/app/games/brickbreakertwo/brickbreakertwo-start-page** – componentă folosită pentru a crea pagina de start a jocului Brick Breaker 2
- **src/app/games/brickbreakertwo/brickbreakertwo1** (**brickbreakertwo2, brickbreakertwo3, brickbreakertwo-heart**) – componente folosite pentru nivele jocului Brick Breaker 2
- **src/app/goalscorer** – componentă folosită pentru dezvoltarea jocului Goal Scorer
- **src/app/goalscorer/goalscorer-start-page** – componentă folosită pentru crearea paginii de start a jocului Goal Scorer
- **src/app/snake/snake-start-page** – componentă folosită pentru crearea paginii de start a jocului Snake
- **src/snake/snake1** (**snake2, snake3, snake-zero-wall**) – componente folosite pentru nivelele jocului Snake
- **src/app/superbricks** – componentă folosită pentru dezvoltarea jocului Super Bricks
- **src/app/goalscorer/superbricks-start-page** – componentă folosită pentru crearea paginii de start a jocului Super Bricks
- **src/app/tetris** – componentă folosită pentru dezvoltarea jocului Tetris
- **src/app/goalscorer/tetris-start-page** – componentă folosită pentru crearea paginii de start a jocului Tetris
- **src/app/games-page** – componentă folosită pentru dezvoltarea paginii unde utilizatorul poate alege jocul pe care îl va juca
- **src/app/login** – componentă creată pentru dezvoltarea paginii de login
- **src/app/rankings** – componentă folosită pentru crearea paginii cu clasamentele pentru fiecare joc
- **src/app/rankings/** (**brickbreaker-rankings, brickbreakertwo-rankings, goalscorer-rankings, snake-rankings, superbricks-rankings, tetris-rankings**) – componente folosite pentru dezvoltarea paginilor cu clasamente pentru fiecare joc
- **src/app/register** – componentă creată pentru dezvoltarea paginii pentru înregistrare
- **src/app/start-page** – componentă utilizată în crearea primei pagini a aplicației

- **src/app/auth.interceptor.ts** – folosit pentru a pune automat în cererile HTTP token-ul JWT
- **src/app/auth.service.ts** – pentru a gestiona sistemul de autentificare din Angular
- **src/app/bug.service.ts** – pentru a gestiona sistemul de reportarea a bug-urilor de către utilizator
- **src/app/chat.service.ts** – pentru a gestiona sistemul de chat (afișarea și trimiterea mesajelor)
- **src/app/score.service.ts** – folosit pentru a gestiona scorurile în joc (afișarea top-ului, clasamentelor, actualizarea scorului)
- **src/environments/environment.ts** – folosit pentru a stoca setările pentru modul de dezvoltare și URL-ul API-ului ASP.NET Core

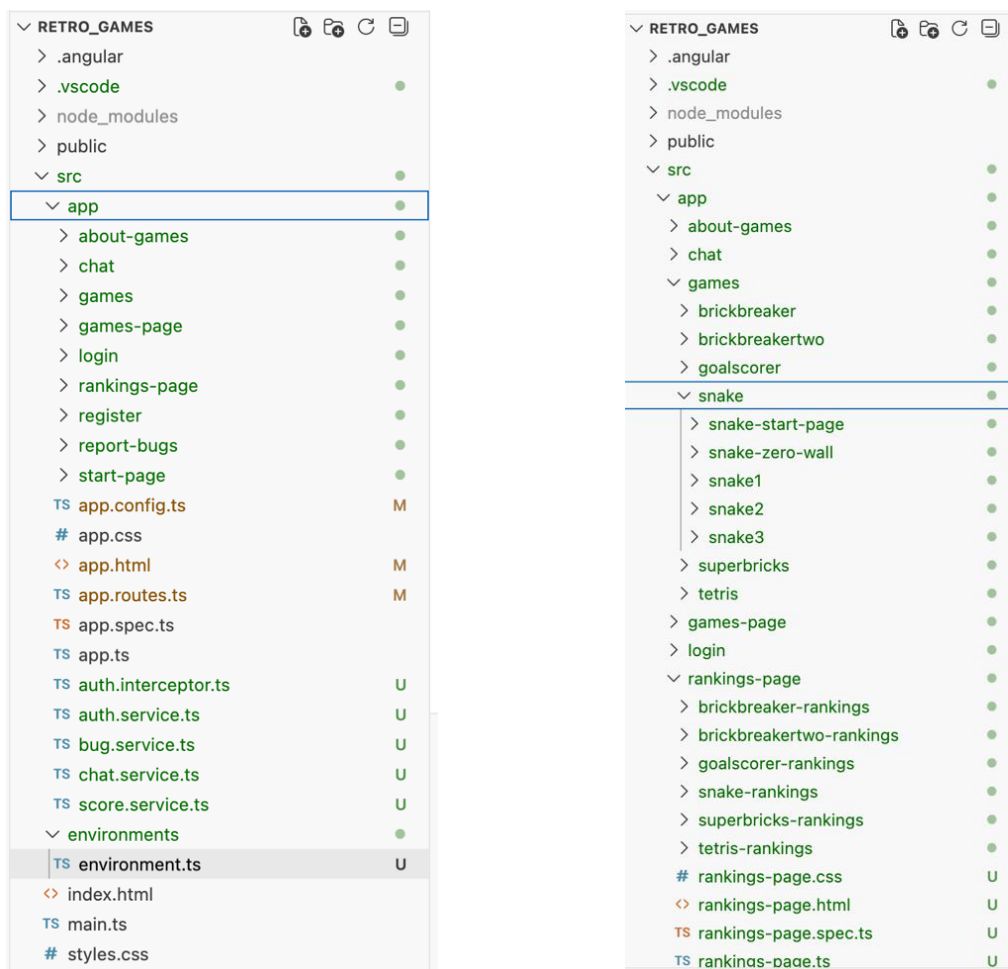


Fig. 16 - Structura proiectului Retro Games (frontend)

4.3.2 Pregătirea mediului de lucru pentru partea de backend

Pentru realizarea părții de backend s-a folosit aplicația „Rider” și s-a creat un nou proiect denumit „Retro_Games_backend”, folosind limbajul de programare C# și șablonul Web API.

În cadrul aplicației backend au fost create următoarele elemente (**Fig. 17**):

- **Controllers** - pentru a asigura punțile de comunicare între frontend și backend. Acestea gestionează cererile HTTP, apelează serviciile necesare și returnează răspunsurile adecvate către client.
- **Models** - pentru a descrie datele din aplicație, respectiv entitățile stocate în baza de date
- **ApplicationDbContext.cs** - în care se realizează conexiunea la baza de date.

După crearea modelelor s-a realizat și prima migrare prin comanda „add-migration”, iar baza de date a fost actualizată prin comanda „update-database”. De fiecare dată când au existat modificări în baza de date s-au realizat noi migrații.

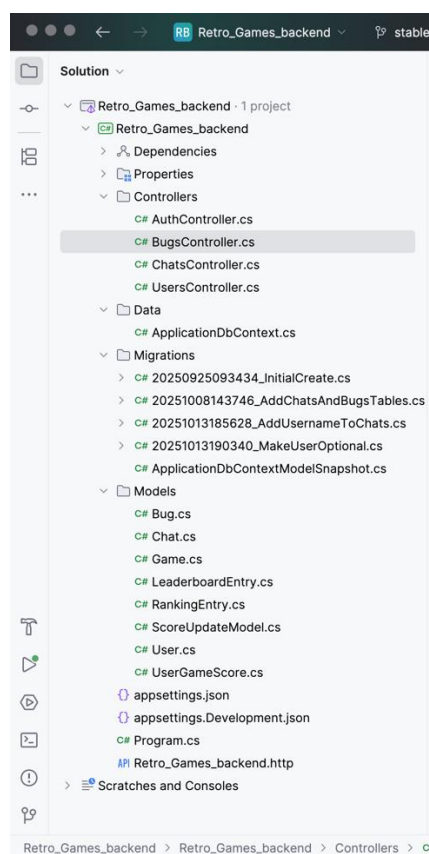


Fig. 17 - Structura proiectului Retro Games (backend)

4.4 PAGINILE DE START, LOGIN ȘI REGISTER

Pagina de start a aplicației (**Fig. 18, Fig. 19**) este formată din titlul platformei și două butoane, pentru autentificare, respectiv înregistrare pe platformă. Codurile ce stau la baza navigării către aceste două pagini vor fi lăsate mai jos, în ordinea prezentată mai sus. Deoarece prima variantă a paginii de start este mult prea simplistă, s-a inserat ca background o imagine de fundal¹ și s-au folosit clasele din Bootstrap pentru stilizarea butoanelor și titlului după cum se poate observa în figurile de mai jos.

¹ <https://pixabay.com/illustrations/retro-images-70s-ads-80s-commercials-9038410/>

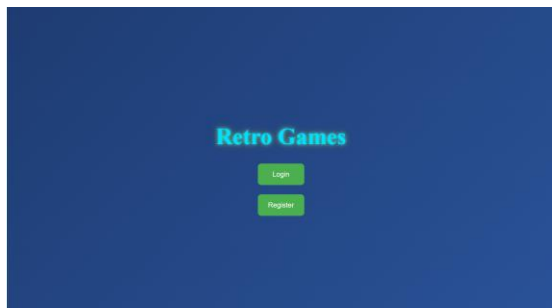


Fig. 18 - Pagina de start (prima variantă)



Fig. 19 - Pagina de start (a doua variantă)

```
goToLogin() {
  this.router.navigate(['/login']);
}

goToRegister() {
  this.router.navigate(['/register']);
}
```

Pentru a permite utilizatorului să se înregistreze/autentifice s-au creat două formulare în care utilizatorul trebuie să își introducă numele de utilizator și parola, respectiv să își confirme parola în vederea înregistrării. Codul pentru formularul de înregistrare este prezentat mai jos.

```
<div class="row">
  <form (ngSubmit)="login()" class="col-md-4 my-4 mx-md-auto text-center">
    <div class="mb-3">
      <label for="username" class="form-label">Username</label>
      <input [(ngModel)]="username" name="username" id="username"
class="form-control">
    </div>

    <div class="mb-3">
      <label for="password" class="form-label">Password</label>
      <input [(ngModel)]="password" type="password" name="password"
id="password" class="form-control">
    </div>

    <button type="submit" class="btn btn-outline-success btn-lg d-block
w-100 btn_login">Login</button>
  </form>
</div>
```

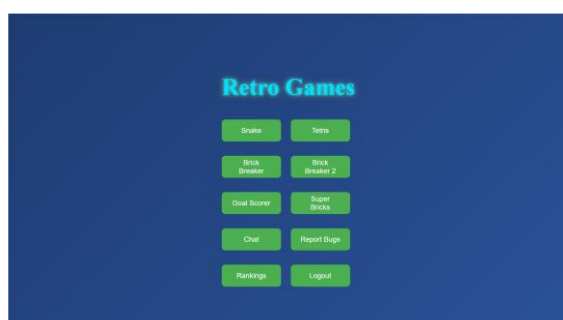
O dată completate aceste informații, la apăsarea butonului „Register” datele vor fi prelucrate în *register.ts*, urmând ca utilizatorul să fie redirecționat către pagina de autentificare.

În cadrul paginii de autentificare există un formular asemănător paginii de înregistrare, diferența constând în neconfirmarea parolei. O dată completate câmpurile și apăsând butonul „Login”, datele vor fi preluate și trimise către server, iar dacă autentificarea a reușit se vor păstra în „Local storage”, username-ul, JWT-ul și id-ul utilizatorului.

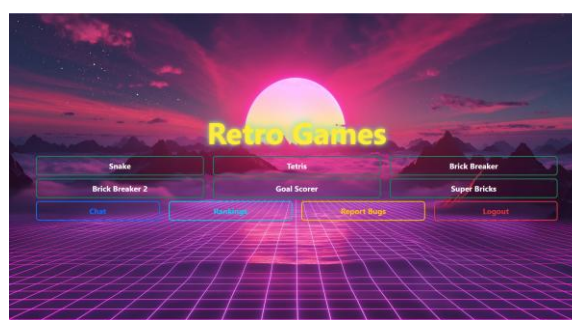
4.5 PAGINA PRINCIPALĂ

Pagina principală (Fig. 20, Fig. 21) este pagina de „administrare” a jocurilor, dar și de redirectionare către pagina pentru chat, raportare a bug-urilor, pagina de clasamente, respectiv permite deconectarea utilizatorului. Pagina principală a fost inițial gândită așa cum se vede în Fig. 20, urmând să se adauge imaginea de fundal² și să se folosească sistemul grilei receptive atât pentru titlu, cât și pentru butoane, acestea fiind și stilizate ulterior folosind clasele din Bootstrap.

Pentru a se realiza redirectionarea către paginile prezentate mai sus s-au folosit rute așa cum se poate observa în script-ul *games-page.ts* ce se găsește în Anexa 2.



*Fig. 20 - Pagina principală
(prima variantă)*



*Fig. 21 - Pagina principală
(a doua variantă)*

4.6 SNAKE

Jocul „Snake” este alcătuit dintr-o pagină principală³ de unde utilizatorul poate alege un mod de joc dintre: „Easy”, „Medium”, „Hard”, „No Wall”, poate vedea detalii despre joc accesând pagina „About the game”, ori se poate întoarce la pagina anterioară.

4.6.1 Modurile de joc

Jocul vine cu 4 moduri de joc, primele 3 se axează pe modificarea vitezei șarpelui și creșterea punctajului în funcție de dificultatea nivelului, iar al patrulea permite șarpelui trecerea prin perete. Datele exacte se pot vedea în tabelul de mai jos (Tabelul 2).

Tabelul 2 Modurile de joc, vizeta de deplasare a șarpelui și punctele obținute

Modul/nivelul de joc	Viteza de deplasare (s)	Puncte
Easy	0,2	1
Medium	0,15	2
Hard	0,1	3
No Wall	0,2	1

² <https://pixabay.com/illustrations/retro-images-70s-ads-80s-commercials-9038411/>

³ <https://pixabay.com/illustrations/snake-reptile-animal-predator-9024888/>

Pentru modul „No wall” s-a modificat funcția *checkBorder()*, astfel încât poziția capului șarpelui să fie resetată corespunzător în funcție de peretele prin care acesta trece.

```
checkBorder() {  
    const last = this.snake[this.snake.length - 1];  
  
    if(last.x >= WIDTH)  
        last.x = 0;  
    else if(last.x < 0)  
        last.x = WIDTH - 40;  
    else if(last.y >= HEIGHT)  
        last.y = 0;  
    else if(last.y < 0)  
        last.y = HEIGHT - 40;  
}
```

4.6.2 Tabla, șarpele și mărul

Tabla de joc (aparența ei fiind modificată ulterior) (**Fig. 22**, **Fig. 23**), s-a desenat ca fiind un dreptunghi cu înălțimea de 800px și lățimea de 600px. Șarpele și mărul au fost reprezentați printr-un câte un array de obiecte de tip „Coordinate”, șarpele având inițial doar patru inele, fiecare de dimensiunea de 40 px, iar mărul fiind un singur punct de dimensiune 40 px.

```
interface Coordinate {  
    x: number;  
    y: number;  
}  
  
apple: Coordinate = {x: 0, y: 0}  
  
snake: Coordinate[] = [  
    {x: 0, y: 280},  
    {x: 40, y: 280},  
    {x: 80, y: 280},  
    {x: 120, y: 280},  
];
```

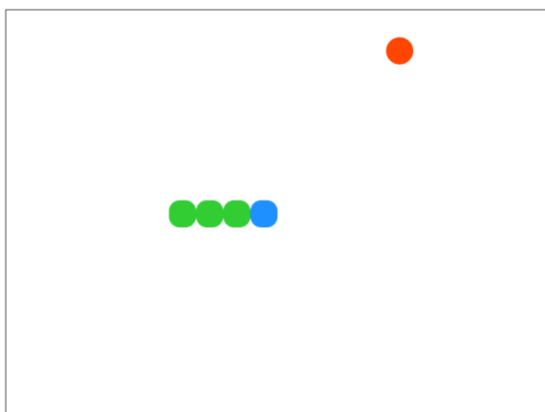


Fig. 22 - Tabla de joc, șarpele și mărul
(prima versiune)

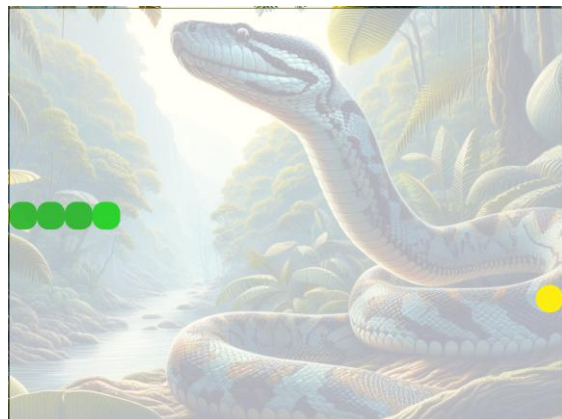


Fig. 23 - Table de joc, șarpele și mărul
(a doua versiune)

4.6.3 Logica de funcționare a jocului

Pentru realizarea jocului „Snake” s-au folosit următoarele funcții (cu mici modificări în funcție de nivel):

- *startInterval()* – pornește intervalul pentru ca șarpele să se miște
- *setInterval()* – în cadrul acestei funcții se verifică dacă jocul a început, dacă șarpele este în mișcare sau jocul este pe pauză, se setează viteza de deplasare a șarpelui, se modifică poziția șarpelui în funcție de direcția aleasă, și se verifică dacă acesta a mâncat mărul, dacă a lovit peretele sau pe el însuși.

```
switch (this.direction) {
  case Direction.RIGHT:
    this.snake[size - 1].x += DELTA
    break;
  case Direction.LEFT:
    this.snake[size - 1].x -= DELTA
    break;
  case Direction.UP:
    this.snake[size - 1].y -= DELTA
    break;
  case Direction.DOWN:
    this.snake[size - 1].y += DELTA
    break;
}

this.checkApple();

this.checkBorder();

this.checkHit();
```

- *checkApple()* – se verifică dacă coordonatele capului șarpelui se suprapun cu cele ale mărului. În caz afirmativ, se adaugă un nou inel șarpelui și se incrementează numărul de puncte în funcție de nivel.

```
checkApple() {
  const last = this.snake[this.snake.length - 1];

  if(last.x == this.apple.x && last.y == this.apple.y) {
    this.snake.unshift({x: this.snake[0].x, y: this.snake[0].y})
    this.points++;
    this.placeApple();
  }
}
```

- *checkHit()* – se verifică dacă coordonatele capului șarpelui se suprapun cu coordonatele oricărui inel al lui. În caz afirmativ se va apela funcția *gameOver()*.

```
checkHit()
{
  const last = this.snake[this.snake.length - 1];

  if(this.snake.find((b) => last != b && b.x === last.x && b.y ==
last.y)) {
    this.message = 'You lost!';
    this.gameOver();
    this.playing = false;
  }
}
```

```
}
```

- *checkBorder()* – În cadrul acestei funcții se va verifica dacă capul șarpelui a depășit tabla de joc în una dintre cele 4 direcții: sus, jos, stânga, dreapta.

```
checkBorder() {  
    const last = this.snake[this.snake.length - 1];  
  
    if(last.x >= WIDTH)  
        last.x = 0;  
    else if(last.x < 0)  
        last.x = WIDTH - 40;  
    else if(last.y >= HEIGHT)  
        last.y = 0;  
    else if(last.y < 0)  
        last.y = HEIGHT - 40;  
}
```

- *placeApple()* – se generează coordonate x și y aleatoriu, se verifică ca acestea să nu intersecteze vreun inel al șarpelui, și în caz afirmativ vor deveni noile coordonate ale mărului.
- *gameOver()* – jocul se va opri și se va verifica dacă există un nou *highscore*, în caz afirmativ acesta va fi modificat și va fi afișat un mesaj pe ecran.
- *startGame()* – în cadrul acestei funcții se vor reseta toate variabilele la valorile inițiale, inclusiv poziția șarpelui, se vor genera noi coordonate pentru măr și se va actualiza clasamentul, respectiv se va porni intervalul.

```
startGame() {  
    this.message = '';  
    this.points = 0;  
    this.snake = [  
        {x: 0, y: 280},  
        {x: 40, y: 280},  
        {x: 80, y: 280},  
        {x: 120, y: 280},  
    ];  
    this.direction = Direction.RIGHT;  
    if(this.playing === true || this.started === true)  
        this.placeApple();  
    this.playing = true;  
    this.started = true;  
    this.pause = false;  
  
    this.scoreService.getLeaderboard(CURRENT_GAME_NAME).subscribe(lb => {  
        this.leaderboard = lb;  
    });  
  
    this.startInterval();  
}
```

- *goToStartPage()* – în interiorul funcției se află ruta către pagina principală a jocului.
- *onLogout()* – se va deconecta utilizatorul și va fi redirecționat către pagina de autentificare.
- *togglePause()* – pentru a pune jocul pe pauză, ori pentru a-l relua.

```
togglePause()  
{
```



```

    if (!this.playing) return;

    this.pause = !this.pause;

    if (this.pause)
        clearInterval(this.intervalId);
    else
        this.startInterval()
}

```

Pe lângă aceste funcții în scriptul *snake-zero-walls.ts* (ori în orice script pentru nivele) se mai găsesc și:

- Decoratorul *HostListener()* – în care se va bloca scroll-ul la apăsarea săgeților, ori a space-ului și se va modifica variabila *direction* în funcție de caz, dacă aceasta nu a mai fost modificată în timpul intervalului.
- Metoda specială *ngOnInit()* – în cadrul căreia se va prelua username-ul din „Local Storage”, se vor prelua clasamentul și highscore-ul jucătorului curent.

4.6.4 Butoanele utilizate în joc

Inițial cele 4 butoane au fost poziționate deasupra tablei de joc (**Fig. 24**), ulterior fiind mai bine stilizate și poziționate în partea stângă a tablei (**Fig. 25**):

- Play/Restart/Play Again – buton ce apelează funcția *startGame()* pentru a începe jocul, a-l reseta în timp ce acesta rulează, ori a-l relua în caz că șarpele a murit.
- Back – apelează funcția *goToStartPage()* pentru a reveni la pagina principală a jocului.
- Logout – apelează funcția *onLogout()* pentru a deconecta utilizatorul și a-l redirecționa către pagina de autentificare.
- Pause/Resume – pentru a pune jocul pe pauză și a-l relua.



Fig. 24 - Butoanele jocului Snake
(prima versiune)



Fig. 25 - Butoanele jocului Snake
(a doua versiune)

4.6.5 Clasamentul jucătorilor (top 5). Informațiile suplimentare

În partea stângă a tablei, deasupra butoanelor vor fi afișate informații precum: scorul, modul de joc și un mesaj de încurajare, iar în cazul în care jucătorul a pierdut, în partea de jos a butoanelor va fi afișat un mesaj corespunzător (**Fig. 26**, **Fig. 27**).

Pentru a obține clasamentul, s-a făcut o cerere către backend pentru a primi username-ul utilizatorilor și cel mai mare scor al acestora.

```

getLeaderboard(gameName: string): Observable<{ username: string;
highScore: number }[]> {
    const url = `${environment.apiUrl}/Users/leaderboard`;

```

```

const requestBody = {gameName: gameName};

return this.http.post<{ username: string; highScore: number }[]>(url,
requestBody);
}

```

În parte de backend se va verifica dacă numele jocului a fost trimis corespunzător, iar în caz afirmativ se vor returna în ordine descrescătoare primii 5 utilizatori în funcție de scorul obținut la jocul „Snake”.

```

[HttpPost("leaderboard")]
public async Task<ActionResult<IEnumerable<LeaderboardEntry>>>
GetLeaderboard([FromBody] GameRequestModel request)
{
    if (string.IsNullOrEmpty(request.GameName))
    {
        return BadRequest("GameName is required");
    }
    var game = await GetGameNameByNameAsync(request.GameName);
    if (game == null)
    {
        return NotFound($"Game '{request.GameName}' not found.");
    }
    var leaderboard = await _context.UserGameScores
        .Where(ugs => ugs.GameId == game.Id)
        .OrderByDescending(ugs => ugs.HighScore)
        .Take(5)
        .Select(ugs => new LeaderboardEntry
        {
            Username = ugs.User.Username,
            HighScore = ugs.HighScore
        })
        .ToListAsync();

    return Ok(leaderboard);
}

```

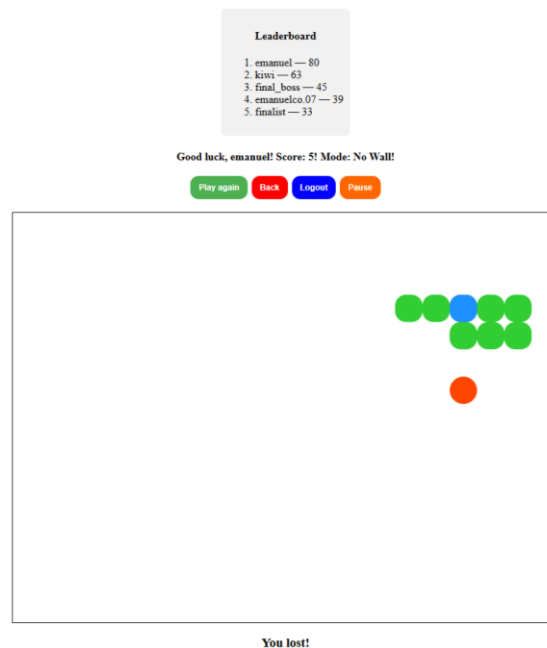


Fig. 26 - Clasamentul și informațiile adiționale (prima versiune)

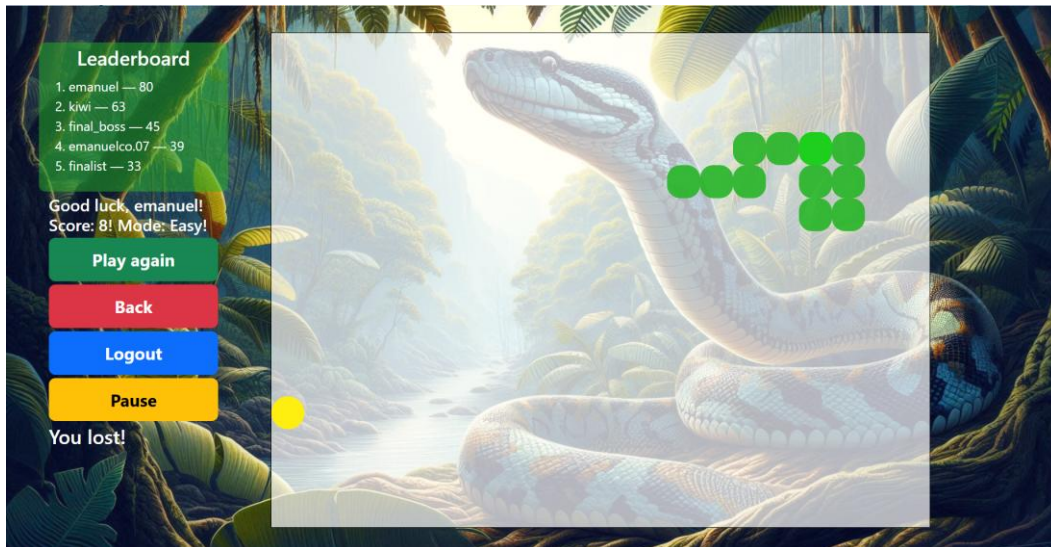


Fig. 27 - Clasamentul și informațiile adiționale (a doua versiune)

4.7 TETRIS

Tetris vine cu o pagină principală⁴ pe care se află butoanele „Play” – pentru a merge către joc, „About the game” – unde se găsesc informații despre joc și regulile acestuia, respectiv butonul „Back” – pentru revenire la pagina principală a jocurilor.

Jocul curent este un Tetris clasic în care dacă piesele sunt rotite și după care lipite de una dintre marginile tablei de joc, acestea trebuie depărtate de margini pentru a putea fi rotite din nou, jocurile de tetris moderne având proprietatea Super Rotation System (SRS) ce permite rotirea pieselor chiar dacă acestea sunt lipite de perete prin mutarea pieselor mai la stânga sau mai la dreapta după caz.

4.7.1 Tabla, piesele (tetromino), următoarea piesă

Tabla de joc este o matrice de 20 de linii și 10 coloane (**Fig. 28, Fig. 29**), fiecare celulă având dimensiunea de 40px. Piesele sunt la rândul lor reprezentate prin matrici de 2x2, 3x3, respectiv 4x4 (pentru piesa I), unde valorile de 1 reprezintă o celulă din piesă ce va fi colorată diferit folosind funcția `getCellColor()`.

```
getCellColor(x: number, y: number): string {
  if (this.arena[y][x] !== 0) {
    return this.arena[y][x] as string;
  }

  const px = x - this.currentPiecePosition.x;
  const py = y - this.currentPiecePosition.y;
  if (
    py >= 0 && py < this.currentPiece.length && px >= 0 &&
    px < this.currentPiece[0].length && this.currentPiece[py][px]
  ) {
    return this.currentPieceColor;
  }
  return '';
}
```

⁴ <https://pixabay.com/illustrations/ai-generated-bricks-tetris-puzzle-8398599/>

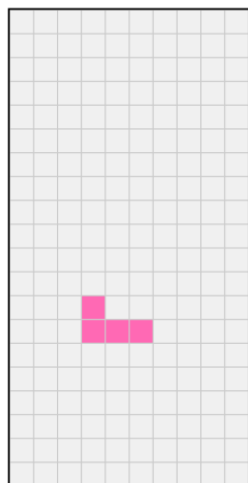


Fig. 28 - Tabla de joc pentru Tetris
(prima versiune)

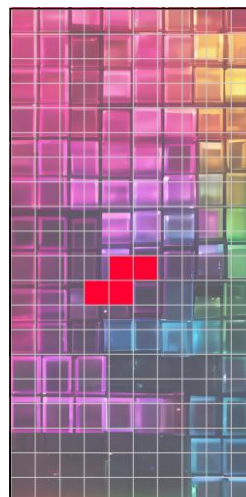


Fig. 29 - Tabla de joc pentru Tetris
(a doua versiune)

Pentru a afișa următoarea piesă (secțiune ce se află în dreapta tablei de joc), s-a folosit o matrice de 4x4 pentru a se putea reprezenta oricare tetromino. Pentru a colora celulele s-a folosit o funcție asemănătoare celei de mai sus, denumită *getNextPieceColor()*, al cărei cod se va găsi în Anexa 4.

Piesele folosite sunt cele prezentate mai jos, reprezentarea lor în matrice fiind asemănătoare cu a piesei T al cărei cod va fi lăsat mai jos.

- Piesa T
- Piesa O
- Piesa I și piesa I rotită
- Piesa S
- Piesa Z
- Piesa L
- Piesa J

```
T_piece: number[][] = [
    [1, 1, 1],
    [0, 1, 0],
    [0, 0, 0]
]
```

Pentru a reține culoarea fiecărei piese s-a declarat o structură de tip dicționar unde fiecărei litere (cheia) ce definește o piesă i s-a atribuit o culoare (valoarea) corespunzătoare, așa cum se poate vedea din codul de mai jos.

```
pieceColors: { [key: string]: string } = {
    T: 'purple',
    O: 'yellow',
```

```

I: 'cyan',
S: 'red',
Z: 'green',
L: 'orange',
J: 'hotpink'
};

```

4.7.2 Logica de funcționare a jocului

Pentru logica de funcționare a jocului „Tetris” s-au folosit următoarele funcții prezentate mai jos:

- *startInterval()* – pornește intervalul pentru ca piesele (tetromino) să înceapă să cadă (vezi Anexa 5 pentru cod).
- *setInterval()* – în această funcție se verifică dacă jocul este în desfășurare, în caz contrar, intervalul se va opri; se verifică dacă piesa curentă s-a lovit de alte piese și în caz afirmativ se va colora partea din arenă corespunzător (folosind culoarea piesei respective), se verifică dacă s-au creat linii întregi care trebuie eliminate, dacă s-a trecut la alt nivel și timpul de cădere al pieselor s-a micșorat iar intervalul trebuie resetat. Tot în caz de coliziune se generează o nouă piesă, ce va fi afișată în secțiunea „Next Piece”, iar piesei curente i se vor da valorile piesei următoare. Se va verifica și dacă jocul s-a încheiat și se va afișa un mesaj corespunzător. În cazul în care nu există coliziune, piesa își va continua căderea. (vezi Anexa 5 pentru cod).
- *movePieceDown()* – funcție pentru a muta piesa în jos (creștem valoarea pe axa y).
- *movePieceLeft()* – funcție pentru a muta piesa în stânga (scădem valoarea pe axa x).
- *movePieceRight()* – funcție pentru a muta piesa în dreapta (creștem valoarea pe axa x).
- *HardDrop()* – metodă folosită pentru a implementa hard drop-ul (coborârea instant) la apăsarea tastei *space*. Cât timp nu există coliziuni cu alte piese, piesa curentă va fi coborâtă și vor fi cumulate câte 2 puncte pentru fiecare celulă pe care aceasta o parcurge.

```

HardDrop() {
    while (!this.checkCollision()) {
        this.points += 2;
        this.movePieceDown();
    }
}

```

- *movePieceClockwise()* – folosită pentru a roti piesele care sunt eligibile în sensul acelor de ceasornic. Piesa O nu se poate roti, iar piesa I are doar două poziții în care poate fi rotită, vertical și orizontal. Piese vor fi rotite doar dacă

nu există coliziuni între ele și alte piese sau pereții tablei. (vezi Anexa 6 pentru cod).

- *getCellColor()* – metodă folosită pentru a obține culoarea celulelor.
- *getNextPieceColor()* – funcție folosită pentru a obține culoarea următoarei piese și a afișa-o în zona „Next Piece”.
- *checkCollision()* – metodă în care se verifică dacă piesa lovește în vreun fel tabla: sus, jos, stânga sau dreapta.

```
checkCollision() {
  for (let py = 0; py < this.currentPiece.length; py++)
    for (let px = 0; px < this.currentPiece[py].length; px++) {
      if (this.currentPiece[py][px]) {
        const x = this.currentPiecePosition.x + px;
        const y = this.currentPiecePosition.y + py;

        if (y + 1 >= this.arena.length)
          return true;
        if (x < 0)
          return true;
        if (x >= this.arena[0].length)
          return true;
        if (y < 0)
          return true;

        if (y + 1 < this.arena.length)
          if (this.arena[y + 1][x] !== 0)
            return true;
      }
    }
  return false;
}
```

- *checkCollisionLeft()* – se verifică dacă celula din stânga este ocupată de altă piesă.
- *checkCollisionRight()* – se verifică dacă celula din dreapta este ocupată de altă piesă.
- *checkDistanceToLeftWall()* – metodă în care se verifică dacă piesa se poate apropia de peretele din stânga. Se verifică cât de aproape sunt valorile 1 din matricea piesei față de perete, pentru a o putea apropia la maxim, deoarece în *currentPiecePosition.x* și *currentPiecePosition.y* se memorează coordonatele celulei aflate pe linia 0 și coloana 0.

- *checkDistanceToRightWall()* – metodă în care se verifică dacă piesa se poate apropia de peretele din dreapta.

```
checkDistanceToRightWall() {
    for (let py = this.currentPiece.length - 1; py >= 0; py--)
        for (let px = this.currentPiece[py].length - 1; px >= 0; px--) {
            if (this.currentPiece[px][py] !== 0)
                return this.currentPiecePosition.x + py < this.arena[0].length
- 1;
        }
    return true;
}
```

- *clearFullRows()* – în cazul în care s-a găsit o linie pe care nu există valori de 0, aceasta va fi eliminată și se va adăuga un nou rând gol la începutul arenei. Totodată se va memora numărul de linii eliminate, care poate fi maxim 4, iar după caz, se va incrementa numărul de puncte ținându-se cont și de nivelul curent. La fiecare 10 linii golite se va incrementa și nivelul.

```
clearFullRows() {
    let contor_for_lines = 0;

    for (let i = this.arena.length - 1; i >= 0;) {
        if (!this.arena[i].includes(0)) {
            this.arena.splice(i, 1);
            this.arena.unshift(new Array(10).fill(0));
            contor_for_lines++;
        }
        else
            i--;
    }

    this.linesCleared += contor_for_lines;
    this.level = Math.floor(this.linesCleared / 10) + 1;

    if (contor_for_lines == 1)
        this.points += 100 * this.level;
    else if (contor_for_lines == 2)
        this.points += 300 * this.level;
    else if (contor_for_lines == 3)
        this.points += 500 * this.level;
    else if (contor_for_lines == 4)
        this.points += 800 * this.level;
```

```

else
    this.points += 0;
}

```

- *getTime()* – funcție folosită pentru a seta viteza cu care coboară piesele în funcție de nivel. La nivelul 1 piesa coboară o celulă la fiecare 0,83 s, la nivelul 6 la fiecare 0,3 s, iar de la nivelul 11 la fiecare 0,05 s.
- *colorArena()* – metodă folosită pentru a colora matricea arenei cu piesa curentă atunci când ea este „așezată”.

```

colorArena() {
    for (let py = 0; py < this.currentPiece.length; py++)
        for (let px = 0; px < this.currentPiece[py].length; px++) {
            if (this.currentPiece[py][px]) { /
                const x = this.currentPiecePosition.x + px;
                const y = this.currentPiecePosition.y + py;

                if (y >= 0 && y < this.arena.length && x >= 0 && x <
this.arena[0].length)
                    this.arena[y][x] = this.currentPieceColor;
            }
        }
}

```

- *generateRandomPiece()* – metodă folosită pentru a genera o nouă piesă fără a se repeta vreuna din ele până când nu s-au generat toate cele 7. În funcție de piesa generată i se vor atribui poziția, culoarea și îi va fi setată frecvența pe 1, pentru a nu mai fi generată din nou. După ce au fost generate toate piesele, vectorul de frecvență va fi setat pe 0. Codul funcției se va găsi în Anexa 7.
- *gameOver()* – folosită pentru a verifica dacă jocul s-a încheiat. În caz afirmativ afișează un mesaj corespunzător și salvează highscore-ul dacă este cazul.
- *startGame()* – funcție în care se inițializează toate variabilele, tabla de joc, tabla unde se afișează piesa următoare, nivelul, se generează piesa curentă și piesa următoare, se apelează funcția *startInterval()*, pentru ca piesele să înceapă să cadă, respectiv se face o cerere către backend pentru a obține clasamentul (top 5).

```

startGame() {
    this.arena = [];
    for (let i = 0; i < 20; i++) {
        this.arena.push(new Array(10).fill(0));
    }
    this.nextPieceArena = [];
    for (let i = 0; i < 4; i++) {

```



```

        this.nextPieceArena.push(new Array(4).fill(0));
    }
    this.playing = true;
    this.pause = false;
    this.level = 1;
    this.startInterval();
    this.generateRandomPiece();
    this.currentPiece = this.workingPiece;
    this.currentPieceColor = this.workingPieceColor;
    this.currentPiecePosition = this.workingPiecePosition;
    this.generateRandomPiece();
    this.nextPiece = this.workingPiece;
    this.nextPieceColor = this.workingPieceColor;
    this.nextPiecePosition = this.workingPiecePosition;
    this.message = '';
    this.points = 0;
    this.scoreService.getLeaderboard(CURRENT_GAME_NAME).subscribe(lb => {
        this.leaderboard = lb;
    });
}

```

- *togglePause()* – pentru a pune jocul pe pauză/a-l relua.
- *goToStartPage()* – pentru redirectionare către pagina principală a jocului.
- *onLogout()* – pentru deconectare.

Pe lângă aceste funcții în *HostListener()*, se blochează scroll-ul la apăsarea tastelor, și se vor apela funcțiile de mutare a piesei curente în funcție de săgeata apăsată, sau funcția *HardDrop()*, dacă se apasă tasta Space. Tetromino-ul va fi mutat doar în cazul în care nu există vreo coliziune (vezi Anexa 8 pentru cod). În *ngOnInit()* se vor obține numele de utilizator al jucătorului, clasamentul și se vor inițializa variabilele cu valorile corespunzătoare, asemănător funcției *startGame()*.

4.7.3 Butoanele utilizate în joc. Clasamentul jucătorilor (top 5). Alte mesaje pentru utilizator.

Asemănător jocului „Snake” și aici sunt prezente 4 butoane (**Fig. 30, Fig. 31**), de data aceasta plasate în partea stângă a paginii, după cum urmează:

- Restart/Play Again – la apăsarea acestora se va apela funcția *startGame()*, pentru a reseta jocul ori a-l relua în caz că utilizatorul a pierdut.
- Back – apelează funcția *goToStartPage()* pentru a reveni la pagina principală a jocului.
- Logout – apelează funcția *onLogout()* pentru a deconecta utilizatorul.

- Pause/Resume – la apăsare se va apela metoda `togglePause()`, pentru a pune pe pauză jocul, ori pentru a-l relua.

Deasupra butoanelor va fi afișat și clasamentul (**Fig. 30**, **Fig. 31**) exact ca la jocul anterior, între ele va fi afișat și un mesaj de încurajare pentru jucător.

Mesajul de final de joc (*You lost!*) va fi și de această dată afișat în partea stângă sub butoane. Scorul și nivelul au fost inițial afișate sub secțiunea „Next Piece” (**Fig. 32**), urmând să fie modificate și afișate între butoane și clasament (**Fig. 31**).

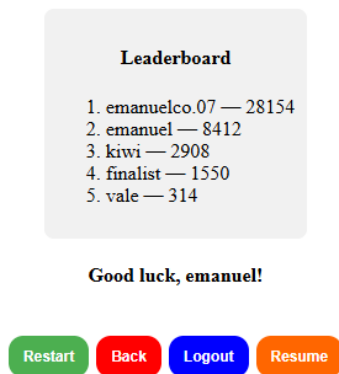


Fig. 30 - Clasamentul, butoanele, mesajul de încurajare (Tetris) (prima versiune)

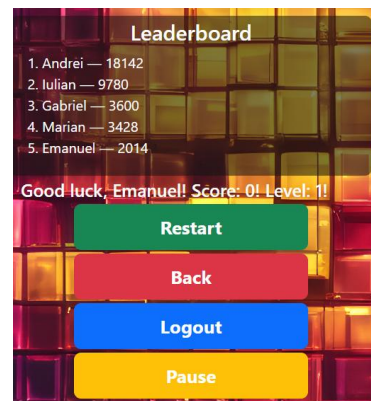


Fig. 31 - Clasamentul, butoanele, mesajul de încurajare (Tetris) (a doua versiune)

4.7.4 Secțiunea afișării piesei următoare „Next Piece”

Pentru a afișa următoare piesă s-au folosit două matrici, una pentru a reprezenta tabla în care se va afișa piesa, a doua pentru a memora forma piesei următoare. În partea dreaptă a ecranului s-a poziționat secțiunea „Next Piece”.

Matricea folosită pentru tablă a fost declarată pentru a putea conține elemente de tip „string”, pentru a memora culoarea celulei piesei, ori valoarea 0 (**Fig. 32**, **Fig. 33**).

```
nextPieceArena: (string | 0)[][] = []
```

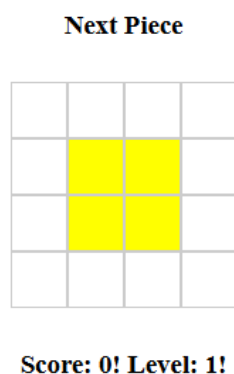


Fig. 32 - Următoarea piesă, scorul, nivelul (prima versiune)

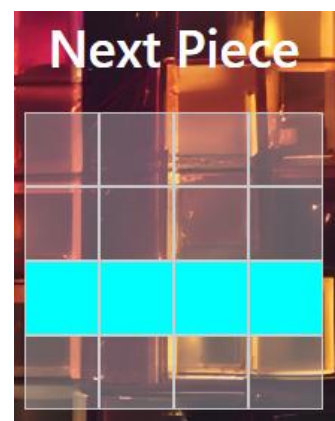


Fig. 33 - Următoarea piesă (a doua versiune)

4.8 BRICK BREAKER ȘI BRICK BREAKER 2

Brick Breaker și Brick Breaker 2 sunt două jocuri ce au la bază distrugerea de ziduri formate din cărămizi cu ajutorul unei bile. Diferența dintre acestea constă în faptul că cel de-al doilea joc are în plus o paletă (paddle) aflată în partea de sus a tablei, astfel încât jucătorul să poată avea mai mult control asupra bilei.

Ambele jocuri vin cu mai multe niveluri de dificultate ce constau în mișcarea mai rapidă a bilei, grosimea mai mare a pereților, iar paleta având mai puține pătrățele (4 pentru cel mai ușor nivel, 3 pentru nivelul mediu și 2 pentru nivelul greu). Ambele vin cu un mod special denumit „Heart Mode” unde zidul este în formă de inimă.

Ambele jocuri au o pagină „About the Game” în care sunt prezentate regulile de joc, cum se joacă, respectiv o mică istorie a jocurilor.

Interfața este aceeași pentru ambele jocuri, indiferent de nivel. În partea stângă se află clasamentul cu primii 5 cei mai buni jucători, mesajul de încurajare, respectiv cele 4 butoane întâlnite la toate jocurile: Restart/Play Again, Back, Logout, Pause/Resume. În partea dreaptă se află secțiunea „Life” în care jucătorul poate vedea câte vieți mai are. De fiecare dată când bila trece pe lângă paletă se va scădea câte o viață. Jocul se încheie atunci când nu mai este nicio viață disponibilă. Sub butoane și clasament este afișat scorul și mesajul „Press Space to Start/Continue!” sau mesajul de final de joc.

4.8.1 Logica de funcționare a jocurilor

Pentru jocurile „Brick Breaker” s-au folosit următoarele funcții prezentate mai jos (se vor prezenta doar pentru primul joc), cu mici diferențe pentru „Brick Breaker 2”:

- *startInterval()* – pornește intervalul pentru ca bila să se poată mișca.
- *setInterval()* – metodă în care verificăm dacă jocul este câștigat sau pierdut, dacă jocul este oprit (pe pauză) sau dacă încă nu s-a apăsat tasta „space”, respectiv verificăm coliziunile bilei cu pereții tablei și zidurile și schimbăm direcția bilei după caz.
- *checkBallCollison()* – funcție în care verificăm dacă bila lovește zidul în partea de sus, stânga sau dreapta; dacă bila lovește paleta și în acest caz îi schimbăm direcția sau dacă trece pe lângă ea, iar atunci o să scădem o viață sau o să oprim jocul dacă nu mai există nicio viață.
- *resetGameRound()* – metodă folosită pentru a reseta poziția bilei, direcția acesteia, poziția paletei, starea tastei „space” ca fiind neapăsată și mesajul ce va fi afișat atunci când se pierde o viață iar jocul poate fi încă continuat.
- *checkWall()* – în cadrul acestei funcții se verifică dacă bila a lovit o cărămidă. Sunt mai multe cazuri ca bila să poată lovi o cărămidă și anume: poate lovi partea de sus, jos, stânga, dreapta a cărămidii sau unul dintre cele 4 colțuri. Pentru a evita orice eroare s-a verificat și cazul în care bila lovește cărămida în centru (se află pe aceeași poziție).
- *destroyBrick()* – metodă în care se distruge cărămida și se incrementează numărul de puncte.
- *reverseY()* – se inversează direcția de mișcare a bilei pe axa Y așa cum se poate vedea în codul de mai jos.

```

reverseY() {
    if (this.direction === Direction.NorthEast) this.direction =
Direction.SouthEast;
    else if (this.direction === Direction.NorthWest) this.direction =
Direction.SouthWest;
    else if (this.direction === Direction.SouthEast) this.direction =
Direction.NorthEast;
    else if (this.direction === Direction.SouthWest) this.direction =
Direction.NorthWest;
    this.BallGoingUp = !this.BallGoingUp;
}

```

- *reverseX()* – folosită pentru a schimba direcția de mișcare a bilei pe axa X.
- *reverseBoth()* – metodă folosită pentru a schimba direcția de mișcare a bilei atunci când aceasta lovește colțul unei cărămizi.
- *movePaddleLeft()* și *movePaddleRight()* – funcții folosite pentru a mișca paleta în partea stângă sau dreaptă. În caz că bila se află pe paletă va fi mișcată o dată cu ea.

```

movePaddleLeft() {
    if (this.PaddlePosition.x > 0)
    {
        this.PaddlePosition.x--;
        if (this.BallPosition.y + 1 === this.PaddlePosition.y &&
            (this.BallPosition.x === this.PaddlePosition.x - 1 ||
            this.BallPosition.x === this.PaddlePosition.x ||
            this.BallPosition.x === this.PaddlePosition.x + 1 ||
            this.BallPosition.x === this.PaddlePosition.x + 2 ||
            this.BallPosition.x === this.PaddlePosition.x + 3 ||
            this.BallPosition.x === this.PaddlePosition.x + 4
            )) {
            this.moveBallWest();
        }
    }

    this.checkBallCollision();
}

```

- *moveBallWest()*, *moveBallEast()*, *moveBallNorthEast()*, *moveBallNorthWest()*, *moveBallSouthEast()*, *moveBallSouthWest()* – metode folosite pentru a schimba direcția bilei spre Vest, Est, Nord-Est, Nord-Vest, Sud-Est și Sud-Vest.
- *getCellColor()* – funcție folosită pentru a obține culoarea celulei specificate.
- *startGame()* – în cadrul acestei metode se inițializează matricea ce reprezintă arena, matricea în care este afișată viața, se resetează variabilele pentru început de joc, poziția paletelor (paletelor) și a bilei, se reconstruiește zidul, se resetează viețile și se obține clasamentul primilor 5 jucători.
- *togglePause()* – pentru a pune jocul pe pauză/a-l relua.
- *goToStartPage()* – pentru redirectionare către pagina principală a jocului.
- *onLogout()* – pentru deconectare.
- *gameOver()* – în interiorul acestei funcții se verifică dacă viața a ajuns pe 0, iar în caz afirmativ se va apela funcția pentru afișarea mesajului de final de joc.

- *gameWon()* – funcție în care se verifică dacă jocul a fost câștigat, adică dacă toate cărămizile au fost distruse.
- *finalGameMessage()* – în cadrul acestei metode se verifică dacă jucătorul și-a depășit highscore-ul și se va afișa un mesaj corespunzător.
- *getLifeArenaColor()* – funcție folosită a afișa câte vieți mai are jucătorul.
- *moveBallFaster()* – funcție folosită pentru a-i permite bilei să se miște mai repede. Aceasta este apelată atunci când este apăsată tasta „space”. Punctajul este incrementat cu un anumit număr de puncte (pe lângă cele obținute la distrugerea cărămizilor) atunci când este apelată această funcție.

În *ngOnInit()* se obține numele utilizatorului, highscore-ul acestuia, respectiv clasamentul cu primii 5 utilizatori, se inițializează arena pentru joc și cea pentru afișarea vieții, respectiv punem variabila *playing* pe true. În *HostListener()* se blochează tastele dacă jocul este oprit sau pus pe pauză, se previne scroll-ul paginii la apăsarea săgeților sau tastei space, respectiv se modifică poziția paletelor după caz.

4.8.2 Ce aduce în plus Brick Breaker 2

Brick Breaker 2 vine în plus cu adăugarea unei noi palete în partea de sus a tablei de joc, așa cum se poate vedea în **Fig. 34** și **Fig. 35**, aceasta se mișcă simultan cu paleta din partea de jos. Datorită acestei modificări zidurile au fost mutate din partea de sus a tablei în partea centrală și s-au realizat câteva modificări la anumite funcții ce vor fi prezentate mai jos.

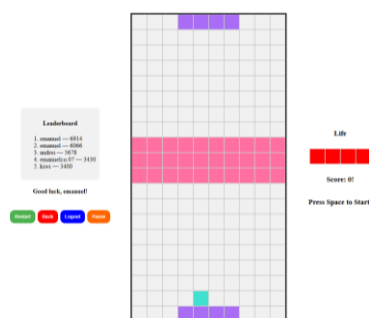


Fig. 34 - Jocul Brick Breaker 2 (prima versiune)

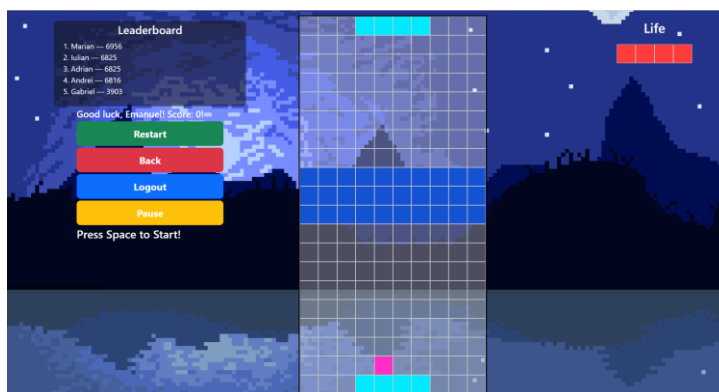


Fig. 35 - Jocul Brick Breaker 2 (a doua versiune)⁵

⁵ <https://pixabay.com/illustrations/mountains-lake-moon-night-water-7728691/>

Modificări:

- În funcțiile *resetGameRound()* și *startGame()* se va reseta și poziția celei de a doua palete.
- Funcțiile *movePaddleLeft()* și *movePaddleRight()* sunt modificate astfel încât să fie mișcate ambele palete.
- În metoda *getCellColor()* se va obține și culoarea paletei de sus.
- Se va pierde câte o viață atunci când bila trece pe lângă oricare dintre cele două palete, modificare făcută în funcția *checkBallCollision()*.

4.9 GOAL SCORER

Goal Scorer (**Fig. 36**, **Fig. 37**) este un joc 2D ce este creat pe același stil precum jocurile precedente. Tema jocului este de a învinge portarul și de a înscrie cât mai multe goluri posibile fără ca să se piardă vieți, adică ca mingea să treacă pe lângă marcator (sau paletă, cum era denumită în celelalte jocuri).

Interfața este aproximativ aceeași cu cea de la jocurile Brick Breaker și Tetris, schimbările apar în cadrul tablei de joc unde în partea de sus se află poarta, formată din doi stâlpi, astfel încât, când mingea se lovește de ei va fi deviată în poartă sau înapoi către atacant; în mijlocul tablei se află portarul care se mișcă constant în partea stângă respectiv dreapta pentru a apăra mingea. În partea de jos se află marcatorul care funcționează exact ca paleta din jocurile Brick Breaker.

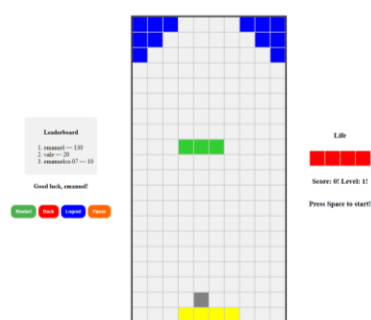


Fig. 36 - Jocul Goal Scorer
(prima versiune)



Fig. 37 - Jocul Goal Scorer (a doua versiune)⁶

Deoarece logica jocului și unele funcții sunt asemănătoare cu cele din jocurile anterioare mai jos vor fi prezentate doar cele esențiale pentru buna funcționare a jocului prezent:

- *checkGoal()* – metodă în care se verifică dacă a fost înscris un gol, adică dacă mingea se află pe rândul 1 (sau 0) al matricei între coloanele 4 și 7 (sau 3 și 6), inclusiv. În cazul în care s-a înscris un gol punctajul va fi mărit cu 10, intervalul se va opri și se vor face resetările necesare precum: poziția mingii, direcția mingii, poziția portarului și a atacantului.
- *GoalkeeperCollision()* – funcție în care se verifică dacă mingea a lovit portarul. În caz afirmativ i se va modifica direcția mingii după caz: se verifică dacă

⁶ <https://pixabay.com/photos/football-pitch-soccer-lawn-320100/>

mingea lovește partea de jos a portarului sau lateralele atunci când se duce în sus (către poartă), respectiv se verifică dacă mingea lovește partea de sus a portarului sau lateralele atunci când mingea se duce în jos (către marcator). Codul acestei funcții poate fi văzut în Anexa 9.

- *checkWallCollision()* – în cadrul acestei metode se verifică dacă bila lovește pereții porții. În caz de coliziune există două posibilități: bila se întoarce înapoi către portar și atacant dacă atinge unul dintre blocurile aflate pe rândurile 1 (doar partea de jos), 2 și 3, respectiv va intra în poartă dacă va atinge partea laterală a unui bloc aflat pe rândul 1.
- *moveGoalkeeper()* – funcție folosită pentru a muta portarul între coloanele 3 și 8 așa cum se poate vedea din codul de mai jos.

```
moveGoalkeeper() {  
    if (this.moveToRight) {  
        if (this.Goalkeeper_Position.x >= 6) {  
            this.moveToRight = false;  
            this.Goalkeeper_Position.x--;  
        } else {  
            this.Goalkeeper_Position.x++;  
        }  
    } else {  
        if (this.Goalkeeper_Position.x <= 2) {  
            this.moveToRight = true;  
            this.Goalkeeper_Position.x++;  
        } else {  
            this.Goalkeeper_Position.x--;  
        }  
    }  
}
```

- *getLevel()* – metodă folosită pentru a incrementa nivelul în funcție de numărul de puncte adunate.
- *getTime()* – funcție folosită pentru a modifica timpul setat în interval în funcție de nivel.

4.10 SUPER BRICKS

În jocul Super Bricks (**Fig. 38, Fig. 39**) se imită o cădere de asteroizi ce trebuie distruși de către o rachetă. Pentru a construi căderea de asteroizi s-a folosit o matrice separată de cea a arenei ce are doar 19 rânduri și la un anumit interval se va genera un nou rând cu valori de 1 pe el (pot exista maxim 6 valori de 1 pe rând pentru a face jocul mai greu), ce va fi adăugat la începutul matricei, iar celelalte rânduri vor fi coborâte mai jos. Se va pierde o viață atunci când există valori de 1 pe rândul 19 al matricei de asteroizi.

Racheta a fost implementată asemănător piesei „T” din jocul Tetris, doar că este rotită la 180 de grade. De fiecare dată când se apasă tasta Space racheta va trage pe coloana pe care se află tunul acesteia și se va adăuga un asteroid pe rândul respectiv.

Rândurile de asteroizi vor fi eliminate atunci când sunt pline, adică dacă pe un rând există doar valori 1. În cazul în care va rămâne un rând gol de asteroizi între alte rânduri cu asteroizi, rândurile aflate în partea de jos a rândului gol, vor fi mutate mai sus.

Interfața jocului este aceeași cu cea de la jocurile anterioare, astfel că în partea stângă se află clasamentul primilor 5 utilizatori, butoanele, mesajul de încurajare, scorul și nivelul, în partea dreaptă fiind afișat numărul de vieți.

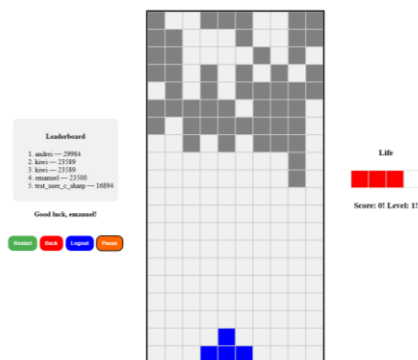


Fig. 38 - Jocul Super Bricks (prima versiune)

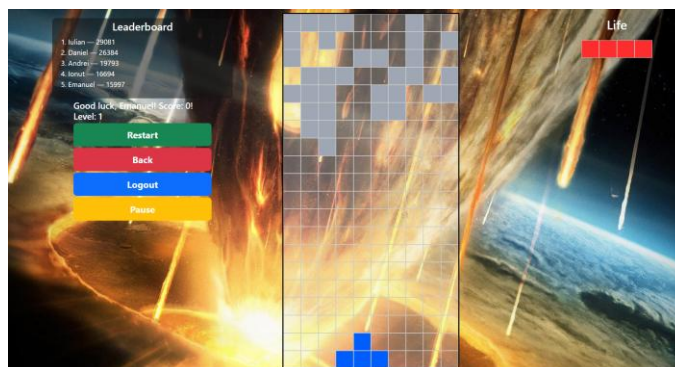


Fig. 39 - Jocul Super Bricks (a doua versiune)⁷

Mai jos vor fi explicate mai în detaliu funcțiile critice ce stau la baza funcționării jocului Super Bricks:

- *Shoot()* – funcție folosită pentru a face racheta să tragă. În cadrul acestei funcții se memorează poziția tunului rachetei și se caută pe coloana respectivă prima celulă eligibilă pentru a se opri proiectilul rachetei în ea. O celulă este eligibilă dacă cea de pe rândul de deasupra ei este ocupată sau dacă s-a ajuns la rândul 0, așa cum se poate observa din codul de mai jos.

```
Shoot() {
    let position = this.RocketPosition.x + 1;
    for(let i = this.Walls.length - 1; i >= 0; i--)
        if(this.Walls[i][position] !== 0)
        {
            this.Walls[i + 1][position] = this.pieceColor['Walls'];
            break;
        }
    else if(i === 0)
    {
        this.Walls[i][position] = this.pieceColor['Walls'];
        break;
    }
}
```

⁷ <https://pixabay.com/photos/meteorite-space-asteroid-cosmos-8691651/>


```
}
}
```

- *checkClearedRows()* – metodă folosită pentru a verifica dacă există un rând plin de asteroizi ce trebuie eliminate. Dacă există, acesta este eliminat și se verifică dacă trebuie mutate rândurile aflate sub el (în cazul în care rămân unul sau mai multe rânduri libere între asteroizi).
- *IfNecessaryMoveRowsUp()* – metodă folosită pentru a muta un rând de asteroizi mai sus atunci când cel de deasupra lui a fost eliminat.
- *generateNewRow()* – funcție ce generează un nou rând de asteroizi și îl adaugă la începutul matricei.

```
generateNewRow() {
    for(let i = 1; i <= 6; i++) {
        let randomNumber: number = Math.floor(Math.random() * 10);
        this.Walls[0][randomNumber] = this.pieceColor['Walls'];
    }
}
```

- *moveWallsDown()* – metodă utilizată pentru a coborî rândurile de asteroizi cu un rând mai jos. Se parcurge matricea de asteroizi de jos în sus (de la valoarea 19 la valoarea 0) și memorăm poziția rândului pe care se află cel puțin un asteroid. Dacă acest rând nu este ultimul rând din matrice, adică rândul 19, atunci vom muta toate celelalte rânduri mai jos. Codul acestei funcții se găsește în Anexa 10.

4.11 PAGINA RANKINGS (CLASAMENTE)

Pagina Rankings (**Fig. 40, Fig. 41**) a fost creată pentru a le permite utilizatorilor să vadă clasamentele întregi de la orice joc, aceasta a fost inițial creată într-un mod minimalist, urmând să fie optimizată pentru diverse dimensiune de ecrane și stilizată folosind Bootstrap, în același stil au fost modificate fiecare dintre paginile individuale pentru clasamentele jocurilor, adăugându-le și lor o imagine de fundal⁸. Pentru a obține clasamentul s-a făcut o cerere către backend, asemănătoare cererii pentru obținerea primilor 5 cei mai buni jucători. Se va face o cerere către baza de date pentru a obține pentru fiecare joc clasamentul ordonat descrescător în funcție de scor.

```
[HttpPost("rankings")]
public async Task<ActionResult<IEnumerable<RankingEntry>>>
GetRankings([FromBody] GameRequestModel request)
{
    if (string.IsNullOrEmpty(request.GameName))
    {
        return BadRequest("GameName is required");
    }
    var game = await GetGameNameByNameAsync(request.GameName);
    if (game == null)
    {
```

⁸ <https://pixabay.com/illustrations/synthwave-cyberpunk-retrowave-cyber-8103557/>

```

        return NotFound($"Game '{request.GameName}' not found.");
    }
    var rankingsRaw = await _context.UserGameScores
        .Where(ugs => ugs.GameId == game.Id)
        .OrderByDescending(ugs => ugs.HighScore)
        .Select(ugs => new
        {
            username = ugs.User.Username,
            highscore = ugs.HighScore
        })
        .ToListAsync();

    var rankings = rankingsRaw
        .Select((entry, index) => new RankingEntry
        {
            Rank = index + 1,
            Username = entry.username,
            HighScore = entry.highscore
        })
        .ToList();

    return Ok(rankings);
}

```

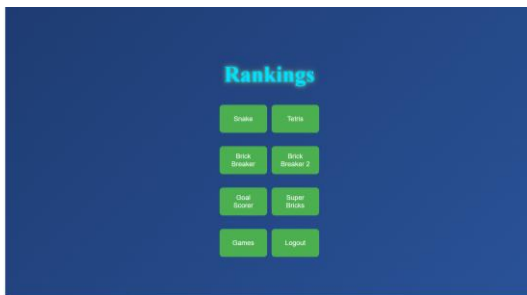


Fig. 40 - Pagina Rankings
(prima variantă)



Fig. 41 - Pagina Rankings (a doua variantă)

4.12 PAGINILE CHAT ȘI REPORT BUGS

Pagina de chat (conversații) (**Fig. 42**, **Fig. 43**) a fost creată pentru a le permite utilizatorilor platformei să discute unii cu alții. Aceasta a fost creată într-un mod minimalist, având un formular în care utilizatorul poate scrie mesajul și un buton pentru a-l trimite. La trimiterea mesajului se va memora în baza de date username-ul utilizatorului care a trimis mesajul, mesajul și data la care a fost trimis, iar mesaje din chat vor fi actualizate pentru a apărea și ultimul mesaj trimis.

```

sendMessage(): void {
    if(this.chatForm.invalid) return;

    const newChat: ChatMessage = {
        username: this.username,
        message: this.chatForm.value.message,

```


4.13 TESTAREA ȘI OPTIMIZAREA APLICAȚIEI

Pe parcursul dezvoltării platformei de jocuri „Retro Games” au fost întâlnite diverse erori ce nu permiteau buna funcționare a anumitor jocuri ori anumite aspecte ale aplicației ce necesitau optimizări. O dată identificate, acestea au fost rezolvate, iar subcapitolul prezent are ca scop expunerea fiecărei erori și soluții ce a fost aplicată pentru ca aplicația să se apropie cât mai mult de idealurile sale de dezvoltare.

În jocul Snake, prima eroare, fiind și cea majoră, consta în faptul că dacă șarpele se mișca într-o anumită direcție, să spunem spre dreapta, și se apăsa tasta cu săgeata spre stânga șarpele își schimba automat direcția spre stânga, practic trecând prin el însuși. Această eroare era valabilă pentru oricare poziție a șarpelui, motiv pentru care codul a fost optimizat pentru a nu mai permite schimbarea direcției de mers a șarpelui dacă aceasta este opusă cu direcția în care el deja se deplasează, adică dacă merge spre stânga să nu poată face dreaptă, dacă merge în jos să nu poată să o ia în sus, ori invers. O altă problemă a jocului Snake ce a necesitat optimizare a fost apariția mărușii, deoarece exista o posibilitate ca acesta să fie plasat pe unul dintre inele șarpelui, a fost necesară o verificare pentru ca acest lucru să nu se întâmple.

La jocurile Brick Breaker 1 și 2, respectiv Goal Scorer o problemă majoră întâlnită era trecerea bilei prin colțurile paletelor sau prin colțurile portarului. După o analiză mai amănunțită a jocurilor de pe consolă am putut observa că bila se lovește inclusiv de colțurile paletelor sau al portarului, motiv pentru care am optimizat codul adăugând o pătrățică invizibilă în stânga și dreapta paletelor pentru ca atunci când bila vine de jos sau sus, să fie deviată când lovește un colț, decizie ce a dus la rezolvarea erorii.

În timpul testării jocului Brick Breaker 1 s-a observat că atunci când se apasă butonul Restart nu mai apar cărămizile puse în formă de inimă, ci apar exact ca la nivelul cel mai greu (hard). Acest lucru era cauzat de o omisiune în funcția de resetare a jocului, unde matricea cărămizilor nu fusese resetată corespunzător.

Deoarece în jocul Goal Scorer era foarte greu de înscris goluri, și pentru că uneori mingea rămânea blocată între portar și pereții porții (adică niciodată nu mai ajungea la marcator, singura soluție de rezolvare a problemei fiind resetarea jocului, caz în care utilizatorul își pierde tot progresul) s-a hotărât ca portarul să ocupe doar două pătrățele și să se miște între coloanele 3 și 8, nu 3 și 7 ca atunci când ocupa 3 poziții. Astfel, rata de succes pentru înscrierea golurilor a crescut și s-a eliminat și cazul în care mingea ar rămâne „blocată”.

În jocurile Brick Breaker 1 și 2, respectiv Goal Scorer bila putea fi mutată și după încheierea jocului prin apăsarea tastei space, motiv pentru care la terminarea jocului tasta space a fost dezactivată, dezactivându-se totodată și celelalte controale pentru a împiedica orice mișcare a componentelor.

Pentru testarea butonul Refresh din pagina Chat s-au deschis două browsere web (Google Chrome și Microsoft Edge), pe fiecare dintre ele autentificarea fiind făcută de pe conturi diferite. S-a trimis un mesaj de pe primul browser (Chrome), iar pe cel de al doilea (Edge) s-a apăsă butonul Refresh pentru a se observa dacă mesajul apare.

5 GHIDUL UTILIZATORULUI

„Retro Games” este o platformă web cu mai multe jocuri retro, majoritatea fiind lansate pentru prima dată încă din anii '90. Scopul aplicației este de a da o nouă înfățișare jocurilor care au stat la baza industriei pe care noi o cunoaștem în prezent.

Platforma numără un total de 6 jocuri și anume: Snake, Tetris, Brick Breaker 1 și 2, Goal Scorer și Super Bricks. Pe lângă acestea în cadrul aplicației se află un chat, pentru a putea discuta cu alți utilizatori, o pagină pentru clasamentele fiecărui joc pentru a vedea cât de bine te clasezi printre ceilalți concurenți și o pagină pentru raportarea erorilor în caz că există o problemă ce trebuie transmisă.

5.1 ÎNREGISTRARE. AUTENTIFICARE. DECONECTARE DIN APLICAȚIE

Pentru a te putea bucura de experiență va trebui prima dată să îți creezi un cont pe platformă. Acest lucru se face într-un mod simplu, deoarece la deschiderea aplicației vei fi întâmpinat cu 2 mari butoane: Login – în caz că ai deja cont și doar vrei să te autentifici folosind numele de utilizator și parola, Register – în caz că ești pentru prima dată pe platformă și ai nevoie să îți creezi cont. Pentru crearea unui cont se va apăsa cel de al doilea buton și vei fi rugat să îți introduci un nume de utilizator, parola și să confirmi parola (memorează-o bine pentru că nu vei avea posibilitatea să o schimbi sau să o recuperezi).

După crearea contului vei fi trimis pe pagina de autentificare, adică exact pagina pe care ai fi intrat dacă ai fi apăsat butonul Login. Acolo vei fi rugat să introduci datele menționate mai sus, după care vei fi redirecționat către pagina cu jocuri.

Pentru deconectare există câte un buton de Logout în cadrul fiecărui joc (sau nivel de joc) și au culoarea albastră, respectiv se pot întâlni pe pagina cu jocuri și pagina cu clasamente având culoarea roșie atunci când cursorul mouse-ului este deasupra lor, altfel fiind transparente și cu margini roșii, ambele fiind responsive, schimbându-și dimensiunile în funcție de rezoluția ecranului pe care sunt afișate. (Fig. 45).



Fig. 45 - Butoanele pentru deconectare (a doua versiune)

5.2 CONTROALELE JOCURILOR

Pentru a putea juca și controla oricare dintre cele 6 jocuri se vor folosi: săgețile stânga, dreapta, sus, jos și tasta Space. Bineînțeles, în fiecare joc acestea vor face lucruri diferite, motiv pentru care controalele vor fi explicate în detaliu mai jos.

5.2.1 Controale Snake

- Săgeata sus – șarpele își va schimba direcția de mers către sus.
- Săgeata jos – șarpele își va schimba direcția de mers către jos.
- Săgeata stânga – șarpele își va schimba direcția de mers către stânga.
- Săgeata dreapta - șarpele își va schimba direcția de mers către dreapta.

5.2.2 Controale Tetris

- Săgeata sus – piesa (tetrominoul) se va roti la 90 de grade în sensul acelor de ceasornic.
- Săgeata jos – tetrominoul va cădea puțin mai repede.
- Săgeata stânga – piesa se va muta o celulă mai în stânga.
- Săgeata dreapta – piesa se va muta o celulă mai în dreapta.
- Tasta Space – piesa se va lipi de primul obstacol pe care îl întâlnește jos (hard drop).

5.2.3 Controale Brick Breaker 1 și 2

- Săgeata stânga – paleta se va mișca în stânga (în caz că bila se află pe paletă se va mișca și ea).
- Săgeata dreapta – paleta se va mișca în dreapta (în caz că bila se află pe paletă se va mișca și ea).
- Tasta Space – bila se va mișca mai repede.

5.2.4 Controale Goal Scorer

- Săgeata stânga – atacantul se va mișca în stânga (în caz că mingea se află la atacant, se va mișca o dată cu el).
- Săgeata dreapta – atacantul se va mișca în dreapta (în caz că mingea se află la atacant, se va mișca o dată cu el).
- Tasta Space – mingea se va mișca mai repede.

5.2.5 Controale Super Bricks

- Săgeata stânga – racheta se va mișca în stânga.
- Săgeata dreapta – racheta se va mișca în dreapta.
- Tasta Space – racheta va trage.

5.3 SCOPUL FIECĂRUI JOC

În fiecare joc scopul este de a aduna cât mai multe puncte, doar că pentru fiecare dintre ele acest lucru se face într-un mod diferit după cum este specificat mai jos:

- Snake: În acest joc trebuie să mănânci cât mai multe mere pentru a aduna puncte. În funcție de nivel, numărul de puncte pe care îl primești atunci când mănânci un măr diferă.
- Tetris: Pentru a aduna puncte trebuie să creezi rânduri pline, ori să cobori piesele folosind hard drop-ul, ori săgeata jos, Cu cât aduni mai multe puncte cu atât nivelul va crește, iar piesele vor cădea mai repede, dar vei și obține mai multe puncte pentru golirea rândurilor.
- Brick Breaker 1 și 2: În aceste jocuri scopul este să distrugi zidurile de cărămizi. Cu fiecare cărămidă pe care o distrugi primești un anumit număr de puncte în

funcție de nivel. Mai primești puncte în plus dacă ți apăsata tasta Space pentru ca bila să se miște mai repede, astfel că pentru fiecare celulă pe care bila o parcurge cât timp tasta Space este apăsată, primești 1 punct.

- Goal Scorer: Obții puncte doar dacă înșcrii goluri, 10 puncte pentru fiecare gol.
- Super Bricks: Asemănător jocului de Tetris primești puncte doar când ai rânduri pline ce pot fi eliminate. Cu cât nivelul este mai mare, cu atât vei primi mai multe puncte pentru fiecare rând de asteroizi eliminat.

5.4 PIERDEREA VIEȚILOR ȘI PIERDEREA JOCULUI

Jocurile nu sunt făcute să meargă la infinit, la un moment dat tot trebuie să se oprească, motiv pentru care există un anumit număr de vieți pe care le poți pierde, iar când rămâi fără viață ai pierdut. Există câteva posibilități de a pierde vieți în funcție de joc, după cum urmează:

- Snake: ai o singură viață pe care o pierzi dacă șarpele se lovește de el însuși, sau dacă șarpele lovește perele (doar în primele 3 nivele, ultimul permițându-i șarpelui să treacă prin perete).
- Tetris: ai o singură viață pe care o pierzi atunci când piesele nu mai au loc să cadă.
- Brick Breaker 1 și 2, Goal Scorer: la aceste jocuri ai 4 vieți pe care le pierzi, una câte una, atunci când bila/mingea trece pe lângă paletă/atacant (marcator).
- Super Bricks: în cadrul acestui joc ai 4 vieți. Pierzi o viață de fiecare dată când rândurile de asteroizi ajung pe rândul pe care se află tunul rachetei.

La jocurile cu mai multe vieți, acestea sunt afișate în partea dreaptă, ca niște pătrățele roșii, așa cum se poate vedea din **Fig. 46**.



Fig. 46 - Afișarea vieții în jocurile cu mai multe vieți

5.5 PAGINILE CHAT, RANKINGS ȘI REPORT BUGS

Pagina de conversații (Chat) poate fi accesată din pagina principală, acolo unde se află butoanele către cele 6 jocuri. O dată intrat pe pagină, vei putea scrie mesaje în secțiunea „Scrie un mesaj...”, iar la apăsarea butonului Trimite mesajul va fi trimis către oricare alt participant, iar chat-ul va fi reîmprospătat astfel încât să vezi ultimul mesaj trimis de tine, dar și dacă alți utilizatori au trimis mesaje. Dacă dorești doar să vezi dacă altcineva a trimis un mesaj, fără ca tu să trimiți un mesaj în prealabil, poți apăsa butonul Refresh (**Fig. 47**) pentru a reîmprospăta secțiunea mesajelor.



Fig. 47 - Butonul Refresh din pagina Chat

Pagina de clasamente (Rankings) poate fi accesată din pagina principală. O dată ajuns pe pagină vei observa butoanele cu numele fiecărui joc, pe care le vei putea folosi pentru a vedea clasamentele întregi ale acestora. La apăsarea oricărui buton, vei fi redirecționat către o pagină unde vei vedea clasamentul de la jocul respectiv. Ca să te întorci la pagina cu jocuri, din pagina de clasamente trebuie apăsat butonul Games (**Fig. 48**).



***Fig. 48** - Butonul Games*

Pagina de raportare a erorilor (Report Bugs) funcționează asemănător paginii Chat, doar că în cazul acesteia, mesajele trimise nu vor apărea pe pagină.

6 CONCLUZII ȘI DEZVOLTĂRI ULTERIOARE

Prin dezvoltarea platformei de jocuri „Retro Games” am reușit bifarea tuturor obiectivelor setate la începerea lucrării. Am creat o aplicație ce readuce în prim-plan jocuri cu care unii dintre noi ne-am născut, am copilărit sau chiar am fost martori la lansarea acestora pe piața jocurilor video.

În continuare voi enumera toate succesele obținute prin crearea aplicației:

- Implementarea a 6 dintre cele mai cunoscute și vechi jocuri 2D din lume și adunarea tuturor într-un singur loc.
- Realizarea unei pagini de chat (conversații) pentru a le permite persoanelor pasionate de jocuri video clasice să comunice unii cu alții.
- Crearea unui mediu competitiv prin prezența clasamentelor pentru fiecare joc, cât și a topului 5 cei mai buni jucători afișat în cadrul fiecărui joc, dar și prin construirea de nivele și creșterea dificultății.
- Elaborarea unei pagini despre fiecare joc (About the Game) pentru a le prezenta utilizatorilor o rază de istorie a jocului respectiv.
- Integrarea fiecărei persoane în comunitatea de jocuri și programare prin crearea paginii Report Bugs (raportarea erorilor) prin care oricare poate comunica ce probleme întâmpină în utilizarea platformei.
- Făurirea unui mediu minimalist prin crearea unei interfețe curate și simple, pe care se află doar butoanele esențiale, cu inscripții sugestive pentru a ușura navigarea și experiența utilizatorului.

În ceea ce privește dezvoltarea aplicației în viitor aș mai dori să lucrez la:

- Adăugarea de noi jocuri mai captivante precum Pac-Man și Flappy Bird.
- Îmbunătățirea paginii de conversații astfel încât mesajele să fie actualizate automat pentru toată lumea atunci când cineva trimite un mesaj, respectiv implementarea scroll-ului automat la ultimul mesaj după ce utilizatorul a apăsă butonul „Trimite” (doar dacă secțiunea „Scrie un mesaj...” nu este goală).
- Rescrierea anumitor bucăți de cod pentru a evita anumite erori ale jocului și pentru a structura mai bine fiecare joc. Crearea de funcții separate pentru fiecare acțiune diferită.
- Organizarea nivelelor într-o singură componentă pentru a elimina fișierele în care există doar mici diferențe.
- Implementarea opțiunii de schimbare a parolei.
- Adăugarea de muzică reprezentativă pentru fiecare joc, pentru ca utilizatorul să aibă parte de o experiență unică, memorabilă atunci când se joacă, să se poată deconecta de mediul extern, perturbat de sunetele cotidiene.
- Optimizarea codului și al tehnologiilor folosite pentru ca timpul de autentificare să fie mai mic.

DECLARAȚIE DE TRANSPARENȚĂ

Secțiunea curentă are ca scop declararea pe propria răspundere a tuturor tehnologiilor, materialelor și resurselor folosite pentru crearea proiectului „Retro Games”, atât a părții scrise, cât și a părții tehnice. Printre acestea se numără asistenți AI precum ChatGPT, Gemini, Copilot și Google AI Studio, respectiv platforma YouTube, ce au fost folosiți pentru structurare, generare și analiză de cod, inspirație, respectiv pentru realizarea cercetării și a găsirii resurselor bibliografice.

Pentru crearea jocului Snake, a primului nivel, cel mai simplu, s-a folosit ca model un tutorial de pe YouTube [22] pentru a înțelege logica jocului, adică cum se poate realiza un „șarpe” care se poate mișca, cum se poate afișa pe ecran, cum se pot verifica coliziunile, cum se poate realiza tabla de joc, ș.a.m.d.

În jocul Tetris, tabla de joc și piesele au fost reprezentate folosind matrici, motiv pentru care a fost nevoie de un alt tutorial [10] pentru a înțelege mai în profunzime cum se reprezintă, cum se verifică coliziunile și cum se afișează piesele colorate. Deoarece aplicația mea folosește limbajul TypeScript, nu JavaScript pentru colorarea pieselor s-a creat o funcție cu ajutorul asistentului AI Copilot [3] denumită *getCellColor()* ce se găsește în fișierul Tetris.ts pentru a putea afișa piesele pe tabla de joc.

Funcția *getCellColor()* a fost folosită și în următoarele jocuri (Brick Breaker 1 și 2, Goal Scorer și Super Bricks), fiind actualizată în funcție de necesități. În plus, având-o ca model, s-au creat și funcțiile *getNextPieceColor()* și *getLifeArenaColor()*.

Pentru crearea părții de backend s-au folosit ChatGPT și Gemini [4], [1] pentru structurarea Modelelor (Models), crearea fișierului *ApplicationDbContext.cs*, realizarea migrațiilor, crearea Controllerelor (Controllers) și structurarea fișierului *Program.cs*. Totodată au fost folosite pentru crearea fișierelor: *login.ts*, *register.ts*, *auth.interceptor.ts*, *auth.service.ts*, *bug.service.ts*, *chat.service.ts*, *score.service.ts* și *environment.ts* în partea de frontend a aplicației.

Pe lângă acestea s-a folosit Gemini pentru inspirație și stilizare, adică pentru găsirea de culori care să se asorteze între ele (de exemplu: între capul șarpelui, inelele lui și mărușul între portar, minge și pereții porții) și s-au folosit prompt-uri asemănătoare cu „Ce culori se potrivesc cel mai bine cu #FF00FF pentru a obține o armonie analoagă?”.

Totodată s-a utilizat Google AI Studio [2] pentru cercetare, adică pentru găsirea de resurse bibliografice de calitate, deoarece majoritatea informației a fost găsită pe Wikipedia și s-a dorit creșterea calității documentului prin integrarea de conținut provenit din resurse verificate, validate, ce sunt eligibile pentru a compune bibliografia unui document academic.

BIBLIOGRAFIE

1. ***, Google, Gemini, <https://gemini.google.com/app?hl=ro>, data accesării: -
2. ***, Google, Google AI Studio, https://aistudio.google.com/prompts/new_chat, data accesării: -
3. ***, Microsoft, Copilot, <https://copilot.microsoft.com/>, data accesării: -
4. ***, OpenAI, ChatGPT, <https://chatgpt.com/>, data accesării: -
5. Angular Team, *What is Angular?*, Angular v17 documentation, <https://v17.angular.io/guide/what-is-angular>, data accesării: 7 ianuarie 2026
6. Ben Henick, *HTML and CSS the good parts*, O'Reilly, 2010, https://timf.upg-ploiesti.ro/cursuri/cursuri/DAW/Cursuri%20si%20resurse%20bibliografice/HTML%20and%20CSS_The%20good%20parts.pdf, data accesării: 3 noiembrie 2025
7. Bootstrap Team, *Overview*, Bootstrap v5.3 documentation, <https://getbootstrap.com/docs/5.3/about/overview/>, data accesării: 25 martie 2026
8. Chris Kohler, *Power Up: How Japanese Video Games Gave the World an Extra Life*, BradyGames, 2005, <https://archive.org/details/poweruphowjapane0000kohl/page/84/mode/2up?q=door+door>, data accesării: 4 ianuarie 2026
9. C-sharpcorner, *C# and its Features*, <https://www.c-sharpcorner.com/article/C-Sharp-and-its-features/>, data accesării: 27 februarie 2026
10. Divine203, Coding TETRIS in 3 minutes, <https://www.youtube.com/watch?v=SgVjRZ90viI&list=WL&index=2>, data accesării: 15 august 2025.
11. Ecma International, *Standard ECMA-334: C# Language Specification. 6th edition*, https://www.ecma-international.org/wp-content/uploads/ECMA-334_6th_edition_june_2022.pdf, data accesării: 27 februarie 2026
12. Evan Amos, *The Game Console: A Photographic History from Atari to Xbox*, No Starch Press, 2019, <https://dokumen.pub/the-game-console-a-photographic-history-from-atari-to-xbox-1593277431-9781593277437.html>, data accesării: 4 ianuarie 2026
13. GeeksforGeeks, *Introduction to TypeScript*, <https://www.geeksforgeeks.org/typescript/introduction-to-typescript/>, 7 ianuarie 2026
14. iD Tech, *The Bad and Good of Flow Free and Bridges*, iD Tech Blog, <https://www.idtech.com/blog/the-bad-and-good-of-flow-free-and-bridges>, data accesării: 31 octombrie 2025
15. JetBrains, *Rider*, <https://www.jetbrains.com/rider/>, data accesării: 4 noiembrie 2025
16. Keza MacDonald, The Guardian, *Tetris Forever is the real story of Tetris – and it's fascinating*, <https://www.theguardian.com/games/2024/nov/13/tetris-forever-is-the-real-story-of-tetris-and-its-fascinating>, data accesării: 31 octombrie 2025
17. MDN Web Docs, *CSS: Cascading Style Sheets*, <https://developer.mozilla.org/en-US/docs/Web/CSS>, data accesării: 7 ianuarie 2026

18. Microsoft, *Prezentare generală a Bootstrap*, Documentație Microsoft Learn, <https://learn.microsoft.com/ro-ro/power-pages/configure/bootstrap-overview>, data accesării: 25 martie 2026
19. Moby Games, *Space Panic*, <https://www.mobygames.com/game/10309/space-panic/screenshots/arcade/705922/>, data accesării: 3 ianuarie 2026
20. MySQL, *What is MySQL*, <https://dev.mysql.com/doc/refman/8.0/en/what-is-mysql.html>, data accesării: 27 februarie 2026
21. Norman J.M., *Steven Russell Programs „Spacewar!”*, the first Computer Game for a Commercially Available Computer, History of information, <https://www.historyofinformation.com/detail.php?id=790>, data accesării: 4 ianuarie 2026
22. Regular SWE, *Can ex-Google engineer code Snake under 60 minutes in Angular 18?*, https://youtu.be/-RwYFm4Mij0?si=s6lXT28_bC28cm-a, data accesării: 15 iulie 2025
23. Snowway22, *Does this count? Fan Art: Nokia 3310, Snake 2, Vector illustration*, https://www.reddit.com/r/retrogaming/comments/12s0gqi/does_this_count_fan_art_nokia_3310_snake_2_vector/, data accesării: 31 octombrie 2025
24. Thomas Hobbs, BBC Culture, *How Pac-Man changed gaming – and the world*, <https://www.bbc.com/culture/article/20250730-how-pac-man-changed-gaming-and-the-world>, data accesării: 4 ianuarie 2026
25. Todd W. Neller, *Pedagogical Possibilities for the 2048 Puzzle Game*, Journal of Computing Science in Colleges, 2015, <https://cupola.gettysburg.edu/cgi/viewcontent.cgi?article=1025&context=csfac>, data accesării: 4 ianuarie 2026
26. Tomohiro Nishikado, Museum of Modern Art, *Space Invaders*, https://www.moma.org/collection/works/169996?artist_id=43700&page=1&sov_refer=artist, data accesării: 4 ianuarie 2026
27. Tony Aguarde, *The Oxford A to Z of Word Games*, Oxford University Press, 1994, https://schooleverywhere-elquds.com/lib/uploadbook/980551479116348The_Oxford_A-Z_of_Word_Games.pdf, data accesării: 3 ianuarie 2026
28. Tony Mott, *1001 Video Games You Must Play Before You Die*, Universe Publishing, 2010, <https://archive.org/details/1001videogamesyo0000unse/page/n9/mode/1up>, data accesării: 3 ianuarie 2026
29. Visual Studio, *Using Git source control in VS Code*, <https://code.visualstudio.com/docs/sourcecontrol/overview>, data accesării: 3 noiembrie 2025
30. Wikipedia, *History of Video Games*, https://en.wikipedia.org/wiki/History_of_video_games, data accesării: 31 octombrie 2025
31. Wikipedia, *List of puzzle video games*, https://en.wikipedia.org/wiki/List_of_puzzle_video_games, data accesării: 31 octombrie 2025
32. Wikipedia, *Puzzle video games*, https://en.wikipedia.org/wiki/Puzzle_video_game, data accesării: 31 octombrie 2025

33. Wikipedia, *Visual Studio Code*, https://en.wikipedia.org/wiki/Visual_Studio_Code, data accesării: 3 noiembrie 2025

ANEXE

ANEXA 1. CODUL SURSĂ PENTRU PAGINA DE ÎNREGISTRARE (REGISTER.TS)

```
<form (ngSubmit)="register()">
  <label>Username:
    <input [(ngModel)]="username" name="username">
  </label><br>
  <label>Password:
    <input [(ngModel)]="password" name="password" type="password">
  </label><br>
  <label>Confirm Password:
    <input [(ngModel)]="confirmPassword" name="confirmPassword"
type="password">
  </label><br>
  <button type="submit">Register</button>
</form>
```

ANEXA 2. CODUL SURSĂ PENTRU PAGINA PRINCIPALĂ (GAMES-PAGE.TS)

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../auth.service';
@Component({
  selector: 'app-games-page',
  imports: [],
  templateUrl: './games-page.html',
  styleUrls: ['./games-page.css']
})
export class GamesPage {
  constructor (private router: Router, private authService: AuthService)
{}
```

```
goToSnake() {
    this.router.navigate(['/snake-start-page']);
}

goToTetris() {
    this.router.navigate(['/tetris-start-page']);
}

goToBrickBreaker() {
    this.router.navigate(['/brickbreaker-start-page']);
}

goToBrickBreaker2() {
    this.router.navigate(['/brickbreakertwo-start-page']);
}

goToGoalscorer() {
    this.router.navigate(['/goalscorer-start-page']);
}

goToSuperBricks() {
    this.router.navigate(['/superbricks-start-page']);
}

goToChat() {
    this.router.navigate(['/chat']);
}

goToReportBugs() {
    this.router.navigate(['/report-bugs']);
}

goToRankingsPage() {
    this.router.navigate(['/rankings-page'])
}

onLogout() {
    this.authService.logout();
}

goToFirstPage() {
    this.onLogout();
}
```

```

    this.router.navigate(['/start-page']);
  }
}

```

ANEXA 3. CODUL PENTRU FUNCȚIA DE REAȘEZARE A MĂRULUI (PLACEAPPLE())

```

placeApple() {
  while (true) {
    const x = Math.floor(Math.random() * GRID_WIDTH) * DELTA;
    const y = Math.floor(Math.random() * GRID_HEIGHT) * DELTA;
    if(this.snake.find((b) => b.x === x && b.y === y)) {
      continue;
    }
    this.apple.x = x;
    this.apple.y = y;
    break;
  }
}

```

ANEXA 4. FUNCȚIA FOLOSITĂ PENTRU A OBȚINE CULOAREA URMĂTOAREI PIESE ÎN JOCLUL TETRIS (GETNEXTPIECECOLOR())

```

getNextPieceColor(x: number, y: number): string {
  let px = x - 0;
  const py = y - 1;
  if (this.nextPiece == this.O_piece)
    px--;
  if (
    py >= 0 && py < this.nextPiece.length && px >= 0 &&
    px < this.nextPiece[0].length && this.nextPiece[py][px]
  ) {
    return this.nextPieceColor;
  }
  return '';
}

```

ANEXA 5. CODUL FUNCȚIILOR PENTRU PORNIREA ȘI SETAREA INTERVALULUI ÎN JOCLUL TETRIS (STARTINTERVAL() ȘI SETINTERVAL())

```

startInterval() {

```



```

if(this.intervalId !== undefined)
    clearInterval(this.intervalId);
this.intervalId = setInterval(() => {
    if (!this.playing)
        return;
    if (this.checkCollision()) {
        this.colorArena();
        this.clearFullRows();
        this.getTime();
        this.startInterval();
        this.currentPiece = this.nextPiece;
        this.currentPieceColor = this.nextPieceColor;
        this.currentPiecePosition = this.nextPiecePosition;
        this.generateRandomPiece();
        this.nextPiece = this.workingPiece;
        this.nextPieceColor = this.workingPieceColor;
        this.nextPiecePosition = this.workingPiecePosition;
        if (this.currentPiece === this.I_piece)
            this.I_pieceRotated = false;
        if (this.gameOver()) {
            this.message = 'You lost!';
            this.playing = false;
            clearInterval(this.intervalId);
        }
    }
    else
        this.movePieceDown();
}, this.time);
}

```

ANEXA 6. CODUL FUNCȚIEI FOLOSITE PENTRU ROTIREA TETROMINO-URILOR ÎN SENSUL ACELOR DE CEASORNIC (MOVEPIECECLOCKWISE())

```

movePieceClockwise() {
    let returnToOriginal: number[][] = this.currentPiece;
    let copyCurrentPiece: number[][] = [];
    if (this.currentPiece !== this.O_piece) {
        if (this.currentPiece === this.I_piece || this.currentPiece ===
            this.I_piece_rotation_2) {

```

```

    if (this.I_pieceRotated === false) {
        this.currentPiece = this.I_piece_rotation_2;
        this.I_pieceRotated = true;
    } else {
        this.currentPiece = this.I_piece;
        this.I_pieceRotated = false;
    }
    copyCurrentPiece = this.currentPiece;
}
else {
    let piecelength = this.currentPiece.length
    for (let i = 0; i < piecelength; i++) {
        copyCurrentPiece.push(new Array(piecelength).fill(0));
    }
    for (let x = 0; x < piecelength; x++)
        for (let y = 0; y < piecelength; y++) {
            copyCurrentPiece[y][piecelength - x - 1] =
this.currentPiece[x][y];
        }
    }
    this.currentPiece = copyCurrentPiece;
    if (this.checkCollision()) {
        this.currentPiece = returnToOriginal;
    }
}
}

```

ANEXA 7. CODUL FUNCȚIEI FOLOSITE PENTRU GENERAREA UNEI NOI PIESE ÎN JOUL TETRIS (GENERATERANDOMPIECE())

```

generateRandomPiece() {
    let randomIndex = Math.floor(Math.random() * 7);
    if (this.piecesFrequency.includes(0)) {
        while (this.piecesFrequency[randomIndex] == 1) {
            randomIndex = Math.floor(Math.random() * 7);
        }
    }
    else {
        this.piecesFrequency = [0, 0, 0, 0, 0, 0, 0, 0];
        randomIndex = Math.floor(Math.random() * 7);
    }
}

```

```

}
switch (randomIndex) {
    case 0:
        this.workingPiece = this.T_piece;
        this.workingPieceColor = this.pieceColors['T'];
        this.workingPiecePosition = { x: 3, y: 0 };
        this.piecesFrequency[0] = 1;
        break;
    case 1:
        this.workingPiece = this.O_piece;
        this.workingPieceColor = this.pieceColors['O'];
        this.workingPiecePosition = { x: 4, y: 0 };
        this.piecesFrequency[1] = 1;
        break;
    case 2:
        this.workingPiece = this.I_piece;
        this.workingPieceColor = this.pieceColors['I'];
        this.workingPiecePosition = { x: 3, y: -1 };
        this.piecesFrequency[2] = 1;
        break;
    case 3:
        this.workingPiece = this.S_piece;
        this.workingPieceColor = this.pieceColors['S'];
        this.workingPiecePosition = { x: 3, y: 0 };
        this.piecesFrequency[3] = 1;
        break;
    case 4:
        this.workingPiece = this.Z_piece;
        this.workingPieceColor = this.pieceColors['Z'];
        this.workingPiecePosition = { x: 3, y: 0 };
        this.piecesFrequency[4] = 1;
        break;
    case 5:
        this.workingPiece = this.L_piece;
        this.workingPieceColor = this.pieceColors['L'];
        this.workingPiecePosition = { x: 3, y: 0 };
        this.piecesFrequency[5] = 1;
        break;
    case 6:
        this.workingPiece = this.J_piece;

```

```

        this.workingPieceColor = this.pieceColors['J'];
        this.workingPiecePosition = { x: 3, y: 0 };
        this.piecesFrequency[6] = 1;
        break;
    }
}

```

ANEXA 8. CODUL FOLOSIT PENTRU A ASCULTA EVENIMENTELE DE LA TASTATURĂ ÎN JOCUL TETRIS (HOSTLISTENER())

```

@HostListener('document:keydown', ['$event'])
keyEvent(event: KeyboardEvent) {
    if(this.pause || !this.playing) return;
    if (
        event.code === 'ArrowUp' || event.code === 'ArrowDown' ||
        event.code === 'ArrowLeft' || event.code === 'ArrowRight' ||
        event.code === 'Space'
    ) {
        event.preventDefault();
    }
    switch (event.code) {
        case KEY_CODE.LEFT_ARROW:
            if (this.checkDistanceToLeftWall() && !this.checkCollision() &&
!this.checkCollisionLeft())
                this.movePieceLeft()
            break;
        case KEY_CODE.RIGHT_ARROW:
            if (this.checkDistanceToRightWall() && !this.checkCollision() &&
!this.checkCollisionRight())
                this.movePieceRight();
            break;
        case KEY_CODE.DOWN_ARROW:
            if (!this.checkCollision()) {
                this.points++;
                this.movePieceDown();
            }
            break;
        case KEY_CODE.UP_ARROW:
            if (!this.checkCollision())
                this.movePieceClockwise();
    }
}

```

```

        break;
    case KEY_CODE.SPACE:
        if (!this.checkCollision())
            this.HardDrop();
        break;
    default:
        break;
    }
}

```

ANEXA 9. CODUL METODEI CE VERIFICĂ COLIZIUNEA DINTRE MINGE ȘI PORTAR ÎN JOCLUL GOAL SCORER (GOALKEEPERCOLLISION())

```

GoalkeeperCollision() {
    if(this.BallGoingUp === true){
        if(this.Ball_Position.y - 1 === this.Goalkeeper_Position.y)
        {
            if(
                this.Ball_Position.x === this.Goalkeeper_Position.x - 1 ||
                this.Ball_Position.x === this.Goalkeeper_Position.x ||
                this.Ball_Position.x === this.Goalkeeper_Position.x + 1 ||
                this.Ball_Position.x === this.Goalkeeper_Position.x + 2
            )
            {
                this.BallGoingUp = false;
                if(this.direction === Direction.NorthEast)
                    return Direction.SouthEast;
                else if(this.direction === Direction.NorthWest)
                    return Direction.SouthWest;
            }
        }

        if(this.Ball_Position.y === this.Goalkeeper_Position.y &&
            this.Ball_Position.x === this.Goalkeeper_Position.x - 1)
        {
            this.BallGoingUp = true;
            if(this.direction === Direction.NorthEast)
                return Direction.NorthWest;
            else if(this.direction === Direction.NorthWest)
                return Direction.NorthWest;
        }
    }
}

```

```

        if(this.Ball_Position.y === this.Goalkeeper_Position.y &&
this.Ball_Position.x === this.Goalkeeper_Position.x + 2)
        {
            this.BallGoingUp = true;
            if(this.direction === Direction.NorthEast)
                return Direction.NorthEast;
            else if(this.direction === Direction.NorthWest)
                return Direction.NorthEast;
        }
    }
    else {
        if(this.Ball_Position.y + 1 === this.Goalkeeper_Position.y)
        {
            if(
                this.Ball_Position.x === this.Goalkeeper_Position.x - 1 ||
                this.Ball_Position.x === this.Goalkeeper_Position.x ||
                this.Ball_Position.x === this.Goalkeeper_Position.x + 1 ||
                this.Ball_Position.x === this.Goalkeeper_Position.x + 2
            )
            {
                this.BallGoingUp = true;
                if(this.direction === Direction.SouthEast)
                    return Direction.NorthEast;
                else if(this.direction === Direction.SouthWest)
                    return Direction.NorthWest;
            }
        }

        if(this.Ball_Position.y === this.Goalkeeper_Position.y &&
this.Ball_Position.x === this.Goalkeeper_Position.x - 1)
        {
            this.BallGoingUp = false;
            if(this.direction === Direction.SouthEast)
                return Direction.SouthWest;
            else if(this.direction === Direction.SouthWest)
                return Direction.SouthWest;
        }

        if(this.Ball_Position.y === this.Goalkeeper_Position.y &&
this.Ball_Position.x === this.Goalkeeper_Position.x + 2)
        {
            this.BallGoingUp = false;
            if(this.direction === Direction.SouthEast)

```

```

        return Direction.SouthEast;
    else if(this.direction === Direction.SouthWest)
        return Direction.SouthEast;
    }
}
return this.direction;
}

```

ANEXA 10. CODUL METODEI CE MUTĂ RÂNDURILE DE ASTEROIZI CU UN RÂND MAI JOS ÎN JOUL SUPER BRICKS (moveWallsDown())

```

moveWallsDown() {
    let row_occupied_position = -1;
    outerLoop:
    for(let i = this.Walls.length - 1; i >= 0; i--)
        for(let j = 0; j < this.Walls[i].length; j++)
            if(this.Walls[i][j] !== 0)
            {
                row_occupied_position = i;
                break outerLoop;
            }
    if(row_occupied_position !== this.Walls.length - 1)
    {
        for(let i = row_occupied_position + 1; i > 0; i--)
            for(let j = 0; j < this.Walls[i].length; j++)
                this.Walls[i][j] = this.Walls[i - 1][j];
        for(let i = 0; i < this.Walls[0].length; i++)
            this.Walls[0][i] = 0;
    }
}
}

```